



TUGAS BESAR

Teori Bahasa Formal dan Otomata



13524007 Muhammad Ashkar
13524039 Rhenaldy Cahyadi Putra
13524069 Miguel Rangga Deardo Sinaga
13524097 Muhammad Faiz Alfada Dharma



DAFTAR ISI

BAB I LANDASAN TEORI.....	3
1.1 Semantic Analysis.....	3
1.2 Abstract Syntax Tree.....	3
1.3 Decorated Abstract Syntax Tree.....	5
1.4 Symbol Table.....	7
1.5 Visitor Function.....	9
1.6 Type Compatibility Rule.....	10
BAB II PERANCANGAN & IMPLEMENTASI.....	11
2.1 Perancangan Program.....	11
2.1.1 Konversi Parse Tree menjadi AST.....	11
2.1.2 Semantic Action.....	13
2.1.3 Symbol Table.....	18
2.1.4 Predefined Identifier.....	20
2.1.5 Type Compatibility Rule.....	22
2.1.6 Decorated AST.....	25
2.1.7 Teknologi yang Digunakan.....	27
2.1.8 Struktur Program.....	27
2.1.9 Fungsi/Modul Utama.....	28
2.1.10 Alur Kerja Program.....	29
2.1.11 Justifikasi Pilihan Implementasi.....	30
2.2 Hasil Implementasi.....	30
BAB III PENGUJIAN.....	32
BAB IV KESIMPULAN & SARAN.....	58
4.1 Kesimpulan.....	58
4.2 Saran.....	58
LAMPIRAN.....	59
REFERENSI.....	59



BAB I LANDASAN TEORI

1.1 Semantic Analysis

Dalam milestone sebelumnya, kita telah mencakup lexical analysis dan syntax analysis. Lexical analysis bertugas mengubah source code menjadi token-token bermakna seperti identifier, keyword, operator, dan literal. Setelah itu, syntax analysis memeriksa apakah urutan token tersebut sesuai dengan grammar bahasa dan membangun Parse Tree.

Namun, program yang lolos syntax analysis belum tentu benar secara makna. Sebagai contoh, pernyataan `x := true + 5` akan lolos parsing karena struktur grammar ekspresi benar, tetapi secara semantik tidak valid karena operasi aritmatika tidak dapat dilakukan antara boolean dan integer. Di sinilah semantic analysis berperan.

Semantic analysis adalah tahap yang memeriksa konsistensi makna program berdasarkan aturan konteks-sensitif bahasa pemrograman. Tahap ini memastikan bahwa setiap identifier telah dideklarasikan sebelum digunakan, tipe data antar operasi kompatibel, parameter prosedur atau fungsi sesuai, penggunaan array benar, serta struktur program sesuai konteks deklarasinya. Dengan kata lain, semantic analysis menjawab pertanyaan: "Apakah program ini masuk akal menurut aturan bahasa?"

Output utama dari semantic analysis pada milestone ini adalah:

1. Abstract Syntax Tree (AST)
Parse Tree dari syntax analyzer masih terlalu detail karena memuat seluruh struktur grammar, termasuk node-node perantara seperti `<term>`, `<factor>`, atau simbol sintaksis seperti tanda kurung. Semantic analyzer menyederhanakan struktur ini menjadi AST yang hanya mempertahankan elemen logis penting.
2. Decorated AST, yaitu AST yang telah diberi anotasi informasi semantik seperti tipe data dan scope,
3. Symbol Table (tab, btab, atab),
Semantic analyzer harus mencatat semua identifier seperti variabel, konstanta, fungsi, prosedur, array, dan block scope ke dalam struktur tabel khusus.
4. Error semantic yang informatif apabila ditemukan pelanggaran aturan makna.

1.2 Abstract Syntax Tree

Setelah semantic analyzer menerima Parse Tree dari tahap syntax analysis, struktur tersebut belum dapat digunakan secara efisien untuk pemeriksaan makna program. Hal ini disebabkan Parse Tree masih menyimpan seluruh detail grammar, termasuk node perantara yang hanya diperlukan selama proses parsing.



Oleh karena itu dilakukan konversi Parse Tree menjadi Abstract Syntax Tree (AST). AST merupakan representasi struktur program yang hanya mempertahankan hubungan logis penting dari program tanpa menyimpan detail sintaks yang tidak lagi diperlukan.

Sebagai contoh, diberikan ekspresi berikut:

```
var
  a, b, c: integer;

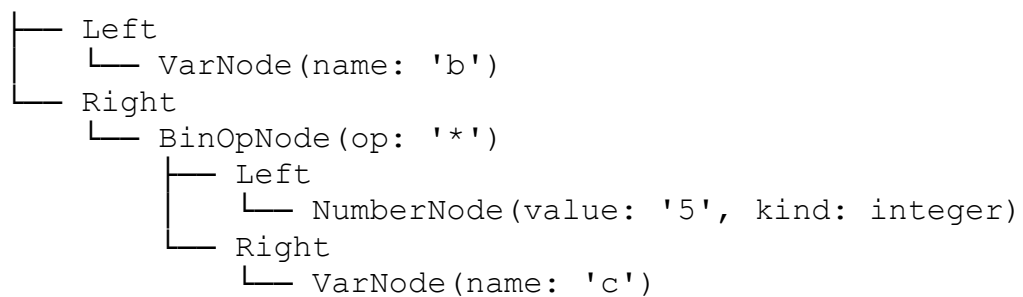
begin
  a := b + 5 * c;
end.
```

Parse Tree yang dihasilkan parser dapat berbentuk:

```
<assignment-statement>
├── <variable>
│   └── IDENT(a)
├── BECOMES
├── <expression>
│   └── <simple-expression>
│       ├── <term>
│       │   ├── <factor>
│       │   │   └── IDENT(b)
│       │   ├── <additive-operator>
│       │   │   └── PLUS
│       │   └── <term>
│       │       ├── <factor>
│       │       │   └── INTCON(5)
│       │       ├── <multiplicative-operator>
│       │       │   └── TIMES
│       │       └── <factor>
│       │           └── IDENT(c)
└──
```

Setelah dikonversi menjadi AST:

```
AssignNode
├── Target
│   └── VarNode(name: 'a')
├── Value
│   └── BinOpNode(op: '+')
└──
```



Terlihat bahwa node seperti <expression>, <term>, <factor>, serta simbol sintaks seperti tanda kurung atau keyword tertentu telah dihilangkan karena tidak lagi diperlukan. Struktur AST menjadi lebih ringkas dan lebih mudah digunakan pada proses semantic analysis maupun code generation.

1.3 Decorated Abstract Syntax Tree

AST yang telah dibentuk pada tahap sebelumnya memang sudah lebih sederhana dibandingkan Parse Tree, tetapi AST tersebut masih hanya menunjukkan struktur logis program. AST belum menyimpan informasi penting yang dibutuhkan untuk memastikan program benar secara semantik.

Oleh karena itu, semantic analyzer melakukan proses anotasi pada setiap node AST. Hasil akhirnya disebut Decorated Abstract Syntax Tree (Decorated AST), yaitu AST yang telah diberi informasi tambahan hasil semantic analysis. Informasi semantik yang ditambahkan pada node dapat berupa: tipe data, kategori identifier, indeks symbol table, hasil type checking, maupun informasi semantik lainnya.

Sebagai contoh, diberikan program yang sama dengan contoh sebelumnya:

```
var
  a, b, c: integer;

begin
  a := b + 5 * c;
end.
```

Setelah dikonversi menjadi AST:



```
AssignNode
├── Target
│   └── VarNode(name: 'a')
├── Value
│   └── BinOpNode(op: '+')
│       ├── Left
│       │   └── VarNode(name: 'b')
│       └── Right
│           └── BinOpNode(op: '*')
│               ├── Left
│               │   └── NumberNode(value: '5', kind: integer)
│               └── Right
│                   └── VarNode(name: 'c')
```

Pada tahap ini semantic analyzer mulai melakukan traversal AST dan melakukan lookup ke symbol table. Dari symbol table diketahui bahwa a, b, c bertipe integer. Semantic analyzer kemudian melakukan pemeriksaan:

1. apakah seluruh identifier sudah dideklarasikan,
2. apakah operasi $5 * c$ valid,
3. apakah operasi $b + (...)$ valid,
4. serta apakah hasil expression kompatibel dengan target assignment a.

Setelah seluruh pemeriksaan selesai, AST diberi anotasi informasi semantic sehingga menjadi Decorated AST:

```
AssignNode(type: void)
├── Target
│   └── VarNode(
│       name: 'a',
│       type: integer,
│       tab_index: 34,
│       level: 0
│   )
├── Value
│   └── BinOpNode(
│       op: '+',
│       type: integer
│   )
│       ├── Left
│       │   └── VarNode(
│       │       name: 'b',
│       │       type: integer,
```



```

    tab_index: 35,
    level: 0
  )
  Right
    BinOpNode(
      op: '*',
      type: integer
    )
    Left
      NumberNode(
        value: '5',
        kind: integer,
        type: integer
      )
    Right
      VarNode(
        name: 'c',
        type: integer,
        tab_index: 36,
        level: 0
      )
    )
  )
)
```

Melalui proses dekorasi ini, setiap node AST tidak hanya merepresentasikan struktur program, tetapi juga menyimpan makna semantic dari program tersebut. Informasi ini sangat penting karena akan digunakan pada proses semantic checking lanjutan maupun tahap code generation.

1.4 Symbol Table

Agar semantic analyzer dapat memahami makna dari setiap identifier yang digunakan pada program, compiler memerlukan struktur penyimpanan khusus yang dapat mencatat informasi mengenai identifier tersebut. Struktur ini disebut Symbol Table.

Symbol Table berfungsi sebagai “database internal” compiler yang menyimpan seluruh identifier beserta atribut pentingnya selama proses semantic analysis berlangsung. Identifier yang dicatat dapat berupa variabel, konstanta, tipe data, procedure, function, parameter, array, maupun record.

Sebagai contoh, diberikan deklarasi berikut:

```
program Hello;
```



```
var
  a, b: integer;

begin
  a := 5;
  b := a + 10;
end.
```

Saat semantic analyzer memproses deklarasi:

```
var
  a, b: integer;
```

identifier a dan b akan dimasukkan ke dalam tab beserta atribut semantic yang diperlukan. Contoh sederhana isi tab setelah deklarasi diproses:

idx	identifier	obj	type	ref	nrm	lev	adr	link
33	Hello	program	void	null	1	0	0	-1
34	a	variable	integer	null	1	0	0	33
35	b	variable	integer	null	1	0	1	34

Selain tab, semantic analyzer juga menggunakan btab untuk menyimpan informasi block atau scope program. Contoh isi btab:

idx	last	lpar	psze	vsze
0	35	0	0	2
1	0	0	0	0

Sedangkan atab digunakan apabila program memiliki tipe array. Tabel ini menyimpan informasi seperti tipe index array, tipe elemen, batas index, dan ukuran array. Ketika semantic analyzer menemukan statement:

```
b := a + 10;
```



maka analyzer akan melakukan lookup identifier a dan b pada symbol table untuk: memastikan identifier telah dideklarasikan, mengambil tipe datanya, mengetahui lexical level identifier, serta membantu proses type checking. Jika identifier tidak ditemukan pada symbol table, semantic analyzer dapat menghasilkan semantic error karena identifier digunakan sebelum dideklarasikan.

1.5 Visitor Function

Setelah AST dan symbol table tersedia, semantic analyzer perlu melakukan pemeriksaan terhadap setiap node pada AST secara terstruktur. Untuk melakukan traversal tersebut digunakan pendekatan Visitor Function.

Visitor Function merupakan mekanisme traversal AST dengan cara mengunjungi setiap node berdasarkan jenis node tersebut. Pada milestone ini, traversal dilakukan menggunakan Depth-First Search (DFS) melalui fungsi visit. Dengan traversal ini, semantic analyzer dapat melakukan lookup identifier, mengelola scope, melakukan type checking, serta memberikan anotasi semantic pada node AST. Sebagai contoh, diberikan AST berikut:

```
AssignNode
├── Target
│   └── VarNode(name: 'b')
└── Value
    └── BinOpNode(op: '+')
        ├── Left
        │   └── VarNode(name: 'a')
        └── Right
            └── NumberNode(value: '10')
```

Saat semantic analysis dilakukan, traversal dimulai dari node paling atas lalu bergerak ke node anak secara DFS. Semantic analyzer terlebih dahulu mengunjungi AssignNode, kemudian memeriksa target assignment berupa VarNode('b'), lalu masuk ke expression BinOpNode('+'). Setelah itu analyzer mengunjungi operand kiri berupa VarNode('a') dan operand kanan berupa NumberNode(10).

Ketika visit(VarNode('a')) dijalankan, semantic analyzer melakukan lookup identifier a pada symbol table untuk memastikan bahwa identifier tersebut sudah dideklarasikan sebelumnya. Jika ditemukan, node akan diberi anotasi seperti tipe data, indeks symbol table, dan lexical level. Selanjutnya pada visit(BinOpNode('+')), semantic analyzer



memeriksa tipe operand kiri dan kanan untuk menentukan apakah operasi + valid serta menentukan tipe hasil expression tersebut.

Traversal dilakukan dari bawah ke atas sehingga hasil pemeriksaan node anak dapat digunakan oleh node induknya. Sebagai contoh, semantic analyzer harus mengetahui terlebih dahulu tipe dari VarNode('a') dan NumberNode(10) sebelum dapat menentukan tipe hasil dari BinOpNode('+').

Selain melakukan type checking, Visitor Function juga digunakan untuk mengelola scope program. Ketika semantic analyzer memasuki block baru seperti procedure, function, atau record, analyzer akan melakukan Push terhadap informasi block pada symbol table. Setelah traversal block selesai, analyzer melakukan Pop sehingga scope kembali ke block sebelumnya.

Dengan pendekatan Visitor Function, proses semantic analysis menjadi lebih terstruktur karena setiap jenis node dapat diproses sesuai tanggung jawab semantic masing-masing. Pendekatan ini juga mempermudah pengembangan compiler ketika grammar atau aturan semantic bertambah pada tahap berikutnya.

1.6 Type Compatibility Rule

Setelah semantic analyzer mampu melakukan traversal AST dan memperoleh informasi identifier dari symbol table, tahap berikutnya adalah memastikan bahwa setiap operasi pada program menggunakan tipe data yang valid. Aturan inilah yang disebut Type Compatibility Rule.

Type Compatibility Rule menentukan bagaimana suatu tipe data dapat digunakan di dalam expression, assignment statement, maupun struktur program lainnya. Pada milestone ini, semantic analyzer melakukan pemeriksaan tipe untuk memastikan bahwa operand dan hasil operasi memiliki hubungan compatibility yang sesuai. Sebagai contoh:

```
a := 5 + 10;
```

merupakan expression yang valid karena kedua operand bertipe Integer. Namun expression berikut:

```
a := true + 5;
```

tidak valid karena operator + tidak dapat digunakan antara Boolean dan Integer. Dengan adanya Type Compatibility Rule, semantic analyzer dapat memastikan bahwa program



tidak hanya benar secara sintaks, tetapi juga benar secara makna dan penggunaan tipe datanya.

BAB II PERANCANGAN & IMPLEMENTASI

2.1 Perancangan Program

2.1.1 Konversi Parse Tree menjadi AST

Setiap non-terminal pada Parse Tree memiliki fungsi konversi tersendiri di dalam ASTBuilder. Berikut adalah pemetaan lengkapnya:

Fungsi ASTBuilder	Node AST yang Dihasilkan	Keterangan
makeProgramNode()	ProgramNode	Ekstrak nama program, deklarasi, dan blok utama dari <program>
makeCompoundNode()	CompoundNode	Kumpulkan semua <statement> dari <statement-list>
makeDeclarationPart()	Beberapa node deklarasi	Delegasikan ke fungsi const/type/var/subprogram sesuai label
makeConstDeclarations()	ConstDeclNode	Iterasi setiap pasangan IDENT = <constant> dengan langkah 4
makeTypeDeclarations()	TypeDeclNode	Iterasi setiap pasangan IDENT = <type> dengan langkah 4
makeVarDeclarations()	VarDeclNode	Iterasi setiap pasangan <identifier-list> : <type> dengan langkah 4



makeSubprogramDeclaration()	SubprogramDeclNode	Bedakan function vs procedure, ekstrak parameter dan return type
makeAssignNode()	AssignNode	children[0] = target variable, children[2] = expression (skip :=)
makeIfNode()	IfNode	children[1]=kondisi, children[3]=then, children[5]=else (jika ada)
makeCaseNode()	CaseNode	Telusuri <case-block> secara rekursif menjadi list CaseBlockNode
makeWhileNode()	WhileNode	children[1]=kondisi, cari <compound-statement> atau <statement>
makeRepeatNode()	RepeatNode	children[1]=<statement-list>, children[3]=kondisi until
makeForNode()	ForNode	Ekstrak iterator, start, arah (to/downto), end, dan blok
makeProcCallNode()	ProcCallNode	children[0]=nama prosedur, cari <parameter-list> untuk argumen
makeVariableNode()	VarNode / ArrayAccessNode / RecordAccessNode	Bangun rantai akses kiri-ke-kanan berdasarkan komponen setelah identifi er
makeExpressionNode()	BinOpNode atau langsung	Jika ada 3 anak: kiri op kanan; jika 1 anak: delegasikan ke simple-expression



makeSimpleExpressionNode()	BinOpNode / UnaryOpNode	Tangani tanda unary, lalu iterasi pasangan (op, term) kiri-ke-kanan
makeTermNode()	BinOpNode atau factor	Iterasi pasangan (op, factor) untuk operator *, /, div, mod, and
makeFactorNode()	Berbagai node	Dispatch ke ProcCall, Variable, Constant, ekspresi dalam kurung, atau not
makeTypeNode()	NamedTypeNode / ArrayTypeNode / RangeNode / EnumNode / RecordTypeNode	Dispatch berdasarkan label child dari <type>
makeConstantNode()	NumberNode / CharNode / StringNode / VarNode	Tangani tanda + / - opsional, lalu dispatch berdasarkan token

2.1.2 Semantic Action

Semantic action adalah tindakan yang dilakukan oleh semantic analyzer pada setiap node AST saat fungsi visit() berjalan secara DFS. Tindakan ini mencakup pencatatan ke symbol table, pengecekan kebenaran semantik, dan anotasi tipe pada node. Pada implementasi ini, semantic action dibagi berdasarkan kategori node AST yang ditangani.

1. Struktur Program

Production Rule	Semantic Action (ASTBuilder)
<code><program> → <program-header> <declaration-part> <compound-statement> period</code>	<pre>progName = strip(header.children[1].label) decls = makeDeclarationPart(children[1]) block = makeCompoundNode(children[2]) program.node = new ProgramNode(progName, decls, block)</pre>



<code><compound-statement> → beginsy <statement-list> endsy</code>	<pre>for child in node.children: if child.label == '<statement-list>': stmts = makeStatementList(child) break compound.node = new CompoundNode(stmts)</pre>
<code><statement-list> → <statement></code>	<pre>stmts = [] for child in node.children: if child.label == '<statement>': n = makeStatementNode(child) if n != null: stmts.append(n) // semicolon dibuang return stmts</pre>

2. Deklarasi

Production Rule	Semantic Action (ASTBuilder)
<code><const-declaration> → constsy</code>	<pre>for i=1; i<size; i+=4: name = strip(children[i].label) val = makeConstantNode(children[i+2]) result.add(new ConstDeclNode(name, val))</pre>
<code><type-declaration> → typesy</code>	<pre>for i=1; i<size; i+=4: name = strip(children[i].label) typeDef = makeTypeNode(children[i+2]) result.add(new TypeDeclNode(name, typeDef))</pre>
<code><var-declaration> → varsy</code>	<pre>for i=1; i<size; i+=4: names = makeIdentifierList(children[i]) typeDef = makeTypeNode(children[i+2]) result.add(new VarDeclNode(names, typeDef))</pre>

3. Definisi tipe

Production Rule	Semantic Action (ASTBuilder)
<code><type> → <array-type> <range> <enumerated> <record-type> ident</code>	<pre>actual = (label=='<type>') ? node.children[0] : node switch actual.label: '<array-type>': return makeArrayTypeNode(actual) '<range>': return makeRangeNode(actual) '<enumerated>': return makeEnumNode(actual)</pre>



	<pre>'<record-type>': return makeRecordTypeNode(actual) default: // ident name = strip(actual.label) return new NamedTypeNode(name)</pre>
<pre><array-type> → arraysy lbrack <range> rbrack ofsy <type></pre>	<pre>idxNode = children[2] if idxNode.label == '<range>': indexType = makeRangeNode(idxNode) elif idxNode.label in ['<type>', '<array-type>', '<enumerated>', '<record-type>']: indexType = makeTypeNode(idxNode) else: indexType = new NamedTypeNode(strip(idxNode.label)) elemType = makeTypeNode(children[5]) return new ArrayTypeNode(indexType, elemType)</pre>
<pre><range> → <constant> period period <constant></pre>	<pre>low = makeConstantNode(children[0]) high = makeConstantNode(children[3]) return new RangeNode(low, high)</pre>

4. Statement

Production Rule	Semantic Action (ASTBuilder)
<pre><assignment-statement> → <variable> becomes <expression></pre>	<pre>target = makeVariableNode(children[0]) value = makeExpressionNode(children[2]) return new AssignNode(target, value)</pre>
<pre><if-statement> → ifsy <expression> thensy <statement> [elsey <statement>]</pre>	<pre>cond = makeExpressionNode(children[1]) thenBlock = makeStatementNode(children[3]) elseBlock = null if node.children.size > 5: elseBlock = makeStatementNode(children[5]) return new IfNode(cond, thenBlock, elseBlock)</pre>
<pre><while-statement> → whilesy <expression> dosy <compound-statement> semicolon</pre>	<pre>for i=2; i<size; i++: if children[i].label == '<compound-statement>': loopBlock = makeCompoundNode(children[i]) break elif children[i].label == '<statement>':</pre>



	<pre> loopBlock = makeStatementNode(children[i]) break return new WhileNode(cond, loopBlock)</pre>
<pre><case-statement> → casesy <expression> ofsy <case-block> endsy</pre>	<pre>cond = makeExpressionNode(children[1]) blocks = [] cur = find '<case-block>' in children while cur != null: blocks.add(makeCaseBlockNode(cur)) cur = find '<case-block>' in cur.children return new CaseNode(cond, blocks)</pre>
<pre><case-block> → { <constant> colon } <statement></pre>	<pre>consts = []; stmt = null for child in node.children: if child.label == '<constant>': consts.add(makeConstantNode(child)) elif child.label == '<statement>': stmt = makeStatementNode(child) return new CaseBlockNode(consts, stmt)</pre>
<pre><procedure/function-call> → ident [lparent <parameter-list> rparent]</pre>	<pre>name = strip(children[0].label) args = [] for child in node.children: if child.label == '<parameter-list>': args = makeParameterList(child) break return new ProcCallNode(name, args)</pre>

5. Variable

Production Rule	Semantic Action (ASTBuilder)
<pre><variable> → ident { <component-variable> }</pre>	<pre>base = strip(children[0].label) curr = new VarNode(base) for i=1; i<size; i++: comp = children[i] first = strip(comp.children[0].label) if first == 'LBRACK': // array access idxs = makeIndexList(comp.children[1]) curr = new ArrayAccessNode(curr,idxs) elif first == 'PERIOD': // record access field = strip(comp.children[1].label) curr = new RecordAccessNode(curr,field) return curr</pre>



<pre><index-list> → <expression> { comma <expression> }</pre>	<pre>for i=0; i<size; i+=2: val = children[i] if val.label == '<index-list>': // rekursif untuk nested idxs += makeIndexList(val) continue type = strip token name if INTCON: idx=new NumberNode(v,false) elif CHARCON: idx=new CharNode(v) else: idx=new VarNode(v) indices.add(idx)</pre>
<pre><parameter-list> → <expression> { comma <expression> }</pre>	<pre>for i=0; i<size; i+=2: expr = makeExpressionNode(children[i]) if expr != null: args.add(expr) return args</pre>

6. Ekspresi

Production Rule	Semantic Action (ASTBuilder)
<pre><expression> → <simple-expression> [<relational-operator> <simple-expression>]</pre>	<pre>left = makeSimpleExpressionNode(children[0]) if size > 2: op = mapRelOp(children[1]) right = makeSimpleExpressionNode(children[2]) return new BinOpNode(op, left, right) return left</pre>
<pre><simple-expression> → [plus minus] <term> { <additive-operator> <term> }</pre>	<pre>idx = 0 if children[0] is PLUS/MINUS: sign = '+' or '-' curr = new UnaryOpNode(sign, makeTermNode(children[1])) idx = 2 else: curr = makeTermNode(children[0]) idx = 1 while idx+1 < size: op = mapAddOp(children[idx]) next = makeTermNode(children[idx+1]) curr = new BinOpNode(op, curr, next) idx += 2 return curr</pre>



<code><term> → <factor> { <multiplicative-operator> <factor> }</code>	<pre>curr = makeFactorNode(children[0]) idx = 1 while idx+1 < size: op = mapMulOp(children[idx]) next = makeFactorNode(children[idx+1]) curr = new BinOpNode(op, curr, next) idx += 2 return curr</pre>
<code><factor> → <procedure/function-call></code>	<pre>return makeProcCallNode(child)</pre>
<code><factor> → <variable></code>	<pre>return makeVariableNode(child)</pre>
<code><factor> → <constant></code>	<pre>return makeConstantNode(child)</pre>
<code><factor> → lparent <expression> rparent</code>	<pre>return makeExpressionNode(children[1])</pre>
<code><factor> → notsy <factor></code>	<pre>operand = makeFactorNode(children[1]) return new UnaryOpNode('not', operand)</pre>
<code><factor> → intcon</code>	<pre>val = strip(token.label) return new NumberNode(val, isReal=false)</pre>
<code><factor> → realcon</code>	<pre>val = strip(token.label) return new NumberNode(val, isReal=true)</pre>
<code><factor> → charcon</code>	<pre>val = strip(token.label) return new CharNode(val)</pre>
<code><factor> → string</code>	<pre>val = strip(token.label) return new StringNode(val)</pre>

2.1.3 Symbol Table

1. Tab

Tabel tab adalah struktur utama symbol table yang menyimpan seluruh identifier dalam program. Pada implementasi ini, tab direpresentasikan sebagai `std::vector<TabEntry>` dengan pointer link antar entri dalam satu scope membentuk linked list.

Setiap elemen tab adalah objek TabEntry dengan atribut berikut:

Atribut	Tipe Data	Deskripsi
identifier	string	Nama identifier, disimpan uppercase (case-insensitive)
link	TabEntry*	Pointer ke identifier sebelumnya dalam scope yang sama (linked list)
obj	SymbolType	Jenis simbol: KEYWORD, CONSTANT, VARIABLE, TYPE, PROCEDURE, FUNCTION
type	BaseType	Tipe dasar: INTEGER, REAL, BOOLEAN, CHAR, STRING, ARRAY, RECORD, dll.
ref	TEntry*	Pointer ke ATabEntry (array) atau BTabEntry (record/subprogram)
nrm	bool	true = variabel normal (by-value), false = parameter by-reference
lev	int	Lexical level: 0 = global, 1 = dalam subprogram, dst.
adr	int	Offset memori untuk variabel, atau nilai untuk konstanta

2. Btab

Tabel btab (block table) menyimpan informasi setiap blok kode dalam program: program utama, prosedur, fungsi, dan definisi tipe record. Setiap blok mewakili satu scope yang memiliki variabel lokal dan parameter tersendiri.

Berikut merupakan struktur BTabEntry:

Atribut	Tipe Data	Deskripsi
blocks	int	Indeks entri blok di dalam btab
last	TabEntry*	Pointer ke identifier terakhir yang dideklarasikan dalam blok ini
lpar	TabEntry*	Pointer ke parameter terakhir (untuk subprogram); nullptr untuk record
psze	int	Total ukuran parameter blok subprogram (dalam unit memori)
vsze	int	Total ukuran variabel lokal yang dideklarasikan dalam blok



3. Atab

Tabel atab (array table) menyimpan informasi detail setiap tipe array yang dideklarasikan dalam program. Setiap kali ArrayTypeNode dikunjungi oleh visitor, satu entri ditambahkan ke atab.

Berikut merupakan struktur ATabEntry

Atribut	Tipe Data	Deskripsi
arrays	int	Indeks entri array di dalam atab
xtyp	BaseType	Tipe indeks array (misal: INTEGER, CHAR, SUBRANGE). Tidak boleh REAL.
etyp	BaseType	Tipe elemen array (misal: INTEGER, REAL, RECORD, ARRAY)
eref	TEntry*	Pointer ke ATabEntry atau BTabEntry jika elemen bertipe komposit
low	int	Batas bawah indeks (misal: 1 pada array[1..10])
high	int	Batas atas indeks (misal: 10 pada array[1..10])
elsz	int	Ukuran satu elemen array (dalam unit memori, dihitung dari getSize())
size	int	Total ukuran array = (high - low + 1) * elsz

2.1.4 Predefined Identifier

Predefined identifier adalah identifier yang sudah terdefinisi di dalam symbol table sebelum program Arion mulai dianalisis. Identifier ini dimasukkan pada saat konstruktor SymbolTable dipanggil sehingga dapat langsung digunakan oleh program tanpa perlu dideklarasikan ulang.

Identifier yang Dimasukkan pada Inisialisasi sebagai berikut:

Identifier	SymbolType	BaseType	Keterangan
AND	KEYWORD	VOID	Operator logika AND
ARRAY	KEYWORD	VOID	Keyword deklarasi array
BEGIN	KEYWORD	VOID	Keyword pembuka blok



CASE	KEYWORD	VOID	Keyword case-statement
CONST	KEYWORD	VOID	Keyword deklarasi konstanta
DIV	KEYWORD	VOID	Operator pembagian integer
DOWNT0	KEYWORD	VOID	Keyword arah for-loop turun
DO	KEYWORD	VOID	Keyword do pada while/for
ELSE	KEYWORD	VOID	Keyword else pada if
END	KEYWORD	VOID	Keyword penutup blok
FOR	KEYWORD	VOID	Keyword for-loop
FUNCTION	KEYWORD	VOID	Keyword deklarasi fungsi
IF	KEYWORD	VOID	Keyword if-statement
MOD	KEYWORD	VOID	Operator modulo
NOT	KEYWORD	VOID	Operator logika NOT
OF	KEYWORD	VOID	Keyword of pada array/case
OR	KEYWORD	VOID	Operator logika OR
PROCEDURE	KEYWORD	VOID	Keyword deklarasi prosedur
PROGRAM	KEYWORD	VOID	Keyword header program
RECORD	KEYWORD	VOID	Keyword deklarasi record
REPEAT	KEYWORD	VOID	Keyword repeat-loop
THEN	KEYWORD	VOID	Keyword then pada if
TO	KEYWORD	VOID	Keyword arah for-loop naik
TYPE	KEYWORD	VOID	Keyword deklarasi tipe
UNTIL	KEYWORD	VOID	Keyword until pada repeat
VAR	KEYWORD	VOID	Keyword deklarasi variabel
WHILE	KEYWORD	VOID	Keyword while-loop
INTEGER	TYPE	INTEGER	Tipe data bilangan bulat
REAL	TYPE	REAL	Tipe data bilangan real
BOOLEAN	TYPE	BOOLEAN	Tipe data logika benar/salah
CHAR	TYPE	CHAR	Tipe data satu karakter
STRING	TYPE	STRING	Tipe data rangkaian karakter



TRUE	CONSTANT	BOOLEAN	Nilai boolean benar
FALSE	CONSTANT	BOOLEAN	Nilai boolean salah

Seluruh identifier di atas dimasukkan menggunakan `insertTab()` saat konstruktor `SymbolTable()` dipanggil. Karena semua predefined identifier berada di level 0 (global), mereka selalu dapat ditemukan oleh `lookup()` dari scope manapun.

Catatan Implementasi

Identifier keyword (AND, ARRAY, BEGIN, dst.) memiliki `BaseType::VOID` karena keyword bukan merupakan entitas bertipe. Identifier tipe (INTEGER, REAL, BOOLEAN, CHAR, STRING) memiliki `SymbolType::TYPE` dan `BaseType` yang sesuai, sehingga saat visitor menemukan `NamedTypeNode` dengan nama tersebut, dapat langsung mengambil tipe datanya melalui `lookup()`. TRUE dan FALSE terdaftar sebagai `SymbolType::CONSTANT` dengan `BaseType::BOOLEAN`, sehingga dapat digunakan langsung sebagai nilai ekspresi Boolean tanpa deklarasi tambahan oleh pengguna.

2.1.5 Type Compatibility Rule

1. Tipe Dasar (Simple Types)

Terdapat lima tipe dasar (simple types)

Tipe	Sumber Token	Keterangan
Integer	intcon	Bilangan bulat positif maupun negatif
Real	realcon	Bilangan desimal (floating point)
Char	charcon	Satu karakter tunggal, diapit petik tunggal
Boolean	true, false, ekspresi relasional, not/and/or	Nilai logika benar atau salah
String	string	Rangkaian karakter, diapit petik tunggal

Selain simple types, terdapat pula tipe terstruktur dan tipe khusus:

Tipe	Keterangan
Array	Kumpulan elemen bertipe sama, diindeks oleh tipe tertentu (index type)
Record	Kumpulan field yang masing-masing dapat bertipe berbeda



Subrange	Rentang nilai dari tipe ordinal, misalnya 1..10 atau 'a'..'z'
Enumerated	Daftar identifier bernilai ordinal yang didefinisikan pengguna, misalnya (Jan, Feb, Mar)

2. Ekspresi dengan Operator

Operator	Syarat Operand	Tipe Hasil	Contoh
+, -, *	Integer op Integer	Integer	3 + 5
+, -, *	Real op Real atau Integer op Real	Real	3.0 + 5
/	Integer atau Real	Real	10 / 3
div, mod	Integer op Integer	Integer	10 div 3
and, or	Boolean op Boolean	Boolean	a and b
not	Boolean	Boolean	not a
==, <>, <, <=, >, >=	Tipe kompatibel	Boolean	a > b

3. Aturan Type Compatibility

No	Kondisi	Contoh	Status
1	Tipe Identik T1 dan T2 adalah tipe yang persis sama.	Integer <-> Integer	Compatible
		Real <-> Real	Compatible
		Char <-> Integer	Tidak Compatible

4. Aturan Assignment Compatibility

Suatu tipe T2 dikatakan assignment-compatible dengan tipe T1 (dapat ditulis T1 := T2) jika memenuhi salah satu dari kondisi berikut.

No	Kondisi	Contoh
----	---------	--------



1	T1 adalah Real dan T2 adalah Integer	$r := 5 \quad \{ r: \text{Real} \}$
2	T1 dan T2 compatible bertipe Integer/Boolean/Char, dan nilai T2 merupakan subset dari nilai T1	$\text{idx} := 5 \quad \{ \text{idx}: [1..10] \}$
3	T1 dan T2 compatible dan bertipe String	$s1 := s2 \quad \{ \text{keduanya String} \}$

5. Aturan Ekspresi Kondisi

Statement	Syarat Tipe Kondisi	Keterangan
if <expr> then	Boolean	Ekspresi kondisi wajib bertipe Boolean
while <expr> do	Boolean	Ekspresi kondisi wajib bertipe Boolean

Berikut contoh ekspresi kondisi dalam bahasa Arion

```

var x : Integer;
begin
  if x then ...           { error: kondisi harus Boolean, bukan
Integer }
  if x > 0 then ...       {valid, ekspresi relasional
menghasilkan Boolean }
  while x do ...          { error: kondisi harus Boolean }
  while x > 0 do ...      { valid }
end;

```

2.1.6 Decorated AST

Decorated AST adalah AST yang telah diberi anotasi informasi semantik pada setiap node-nya. Pada implementasi ini, anotasi disimpan dalam tiga atribut di kelas ASTNode: `exprType` (tipe data node), `tabIndex` (indeks ke entri tab), dan `lev` (lexical level). Anotasi dicetak menggunakan fungsi `printAST()` dari file `astprinter.cpp`.

Setiap jenis node memiliki informasi anotasi yang berbeda sesuai peran semantiknya. Berikut adalah format anotasi per jenis node yang dicetak oleh `astprinter.cpp`:



Node	Format Anotasi yang Dicitak	Atribut Utama
ProgramNode	ProgramNode(name: 'Hello')	programName
CompoundNode	CompoundNode	-
ConstDeclNode	ConstDeclNode(name: 'MAX')	constName + child value
TypeDeclNode	TypeDeclNode(name: 'Point')	typeName + child typeDef
VarDeclNode	VarDeclNode(names: [a, b])	varNames + child typeDef
ParamNode	ParamNode(names: [x], isVarParam: false)	paramNames, isVarParam
SubprogramDeclNode	SubprogramDeclNode(kind: function, name: 'Fib')	isFunction, subprogramName
AssignNode	AssignNode	child target + child value
IfNode	IfNode	child condition/then/else
WhileNode	WhileNode	child condition + loopBlock
RepeatNode	RepeatNode	child statements + condition
ForNode	ForNode(iterator: 'i', direction: to)	iteratorName, isDownto
CaseNode	CaseNode	child condition + cases
CaseBlockNode	CaseBlockNode	child constants + statement
ProcCallNode	ProcCallNode(name: 'writeln')	procName + child arguments
VarNode	VarNode(name: 'a')	name
ArrayAccessNode	ArrayAccessNode	child arrayVar + indices
RecordAccessNode	RecordAccessNode(field: 'x')	fieldName + child recordVar
BinOpNode	BinOpNode(op: '+')	op + child left/right



UnaryOpNode	UnaryOpNode(op: 'not')	op + child operand
NumberNode	NumberNode(value: '5', kind: integer)	value, isReal
CharNode	CharNode(value: 'a')	value
StringNode	StringNode(value: Hello)	value
RangeNode	RangeNode	child lowerBound + upperBound
ArrayTypeNode	ArrayTypeNode	child indexType + elementType
RecordTypeNode	RecordTypeNode	child fields
EnumNode	EnumNode(values: [Jan, Feb, Mar])	identifiers

Berikut adalah contoh Decorated AST yang dihasilkan untuk program sederhana berikut:

```
program Hello;
var
  a, b: integer;
begin
  a := 5;
  b := a + 10;
  writeln('Result = ', b);
end.
```

Decorated AST yang dihasilkan oleh printAST():

```
\-- ProgramNode(name: 'Hello')
  |-- Declarations
  |   \-- VarDeclNode(names: [a, b])
  |       \-- NamedTypeNode (integer)
  |-- Block
  |   |-- AssignNode
  |   |   |-- Target
  |   |   |   \-- VarNode(name: 'a')
  |   |   \-- Value
  |   |       \-- NumberNode(value: '5', kind: integer)
  |   |-- AssignNode
  |   |   |-- Target
  |   |   |   \-- VarNode(name: 'b')
  |   |   \-- Value
  |   |       \-- BinOpNode(op: '+')
```



```
        |          |-- Left
        |          |   |-- VarNode(name: 'a')
        |          |-- Right
        |          |   |-- NumberNode(value: '10', kind:
integer)
        |-- ProcCallNode(name: 'writeln')
        |   |-- StringNode(value: Result = )
        |   |-- VarNode(name: 'b')
```

2.1.7 Teknologi yang Digunakan

Implementasi semantic analyzer ini dikembangkan menggunakan bahasa pemrograman C++ sebagai kelanjutan dari milestone-milestone sebelumnya. Program memanfaatkan Standard Template Library (STL), yaitu `std::vector` dan `std::unique_ptr`, untuk mengelola struktur data symbol table, node-node AST, dan kepemilikan memori secara otomatis.

2.1.8 Struktur Program

```
\---JKW-Tubes-IF2224-2026
|   .gitignore
|   main.exe
|   Makefile
|   README.md
|
+---doc
|   .gitkeep
|   Laporan-1-JKW.pdf
|   Laporan-2-JKW.pdf
|   Laporan-3-JKW.pdf
|
+---src
|   |   main.cpp
|   |
|   +---lexer
|   |   |   lexer.cpp
|   |   |   lexer.hpp
|   |   |   reader.cpp
|   |   |   reader.hpp
|   |   |   token.hpp
|   |
|   +---parser
|   |   node.hpp
```

```

|      |      parser.cpp
|      |      parser.hpp
|      |      treeprinter.cpp
|      |      treeprinter.hpp
|      |
|      +---semantic
|      |      astnode.hpp
|      |      astnode.cpp
|      |      astprinter.hpp
|      |      astprinter.cpp
|      |      asttree.hpp
|      |      asttree.cpp
|      |      astvisitor.hpp
|      |      astvisitor.cpp
|      |      symboltable.hpp
|      |      symboltable.cpp
|      |
|      \---utils
|           exception.hpp
|           utils.cpp
|           utils.hpp
|
\---test
    +---milestone-1
    |
    +---milestone-2
    |
    +---milestone-3
    |
    \---milestone-4

```

2.1.9 Fungsi/Modul Utama

1. Main Program

Main program berperan sebagai penghubung utama seluruh sistem dengan mengatur alur eksekusi program dari awal hingga akhir. Bagian utama program diawali dengan meminta input berupa path dari pengguna, kemudian menggunakan modul reader untuk membuka file tersebut. Selanjutnya, program menjalankan proses lexical analysis untuk menghasilkan daftar token yang sesuai dengan spesifikasi bahasa Arion. Token-token hasil lexer kemudian diproses oleh parser untuk dilakukan analisis sintaks serta pembentukan Parse Tree. Apabila parse tree valid (tidak ada error node), membangun AST melalui ASTBuilder, lalu menelusuri AST dengan ASTVisitor untuk melakukan semantic analysis. 🐾

2. Lexer & Parser

Modul ini berasal dari milestone sebelumnya dengan beberapa perbaikan minor (sesuai dengan Revisi Milestone II). Lexer menggunakan pendekatan DFA untuk tokenisasi, sedangkan Parser menggunakan Recursive Descent Parsing.

3. Semantic Analysis (AST & SymbolTable)

Modul semantic analysis terdiri dari `astnode`, `asttree`, `astvisitor`, `symboltable`, dan `astprinter`. `asttree` mengonversi Parse Tree menjadi AST ringkas dengan Syntax-Directed Translation menggunakan node-node yang didefinisikan di `astnode`, masing-masing mewarisi `ASTNode` dengan atribut dekorasi semantik. `astvisitor` menelusuri AST secara DFS untuk melakukan registrasi simbol, lookup, dan anotasi node menggunakan `symboltable` yang mengimplementasikan tabel `tab`, `btabs`, dan `atab` beserta predefined identifier. Error semantik dilempar sebagai `SemanticError` atau `IdentRedeclarationError`. Hasil akhir dicetak ke terminal melalui `astprinter`.

2.1.10 Alur Kerja Program

Alur kerja program adalah sebagai berikut:

1. **Input:** Main program membaca path file kode sumber.
2. **Lexical Analysis** (Milestone 1): Lexer menghasilkan daftar token;
3. **Syntax Analysis** (Milestone 2): Parser membangun Parse Tree menggunakan Recursive Descent Parsing. Hasilnya dicetak ke terminal dan disimpan ke `test/milestone-2/output.txt`.
4. **AST Construction** (Milestone 3): `ASTBuilder` mengonversi Parse Tree menjadi AST dengan Syntax-Directed Translation, menghasilkan tree yang ringkas dan berorientasi semantik.
5. **Semantic Analysis** (Milestone 3): `ASTVisitor` menelusuri AST secara DFS, melakukan:
 - Inisialisasi symbol table dengan reserved words dan predefined identifier.
 - Registrasi dan lookup simbol.
 - Pengecekan tipe data dan kompatibilitas assignment.
 - Anotasi setiap node.
6. **Output:** Decorated AST dicetak ke terminal melalui `printDecoratedAST()` dan disimpan ke `test/milestone-3/output.txt`. Jika terjadi error semantik, pesan error informatif ditampilkan.

2.1.11 Justifikasi Pilihan Implementasi

Implementasi dirancang dengan memisahkan fase pembangunan AST (`ASTBuilder`) dari fase anotasi/pengecekan (`ASTVisitor`). Pemisahan ini mengikuti prinsip separation of concerns: `ASTBuilder` hanya bertanggung jawab atas transformasi struktural dari Parse Tree ke AST, sedangkan `ASTVisitor` bertanggung jawab atas seluruh logika semantik. Hal ini membuat kode lebih mudah diuji, dikembangkan secara terpisah, dan mudah diikuti alurnya. Pembagian komponen ini juga menjaga struktur program tetap modular,



sehingga implementasi lebih terorganisasi, mudah diuji, serta sesuai untuk mendukung kebutuhan milestone saat ini maupun pengembangan tahap berikutnya.

2.2 Hasil Implementasi

1. ASTNode

ASTNode merupakan kelas dasar untuk seluruh node pada Abstract Syntax Tree, seperti ProgramNode, VarDeclNode, AssignNode, BinOpNode, VarNode, dan node lainnya. Setiap node turunan menyimpan informasi penting dari program yang sudah diringkas dari parse tree, sehingga simbol sintaksis yang tidak dibutuhkan seperti keyword, tanda kurung, dan semicolon tidak lagi disimpan. Pada milestone ini, ASTNode juga digunakan sebagai dasar untuk decorated AST karena memiliki atribut semantic seperti `exprType`, `tabIndex`, dan `lev`. Atribut `exprType` menyimpan tipe hasil ekspresi atau tipe node, `tabIndex` menyimpan indeks referensi ke symbol table, sedangkan `lev` menyimpan lexical level atau scope tempat node tersebut berada.

```
class ASTNode {
public:
    BaseType exprType = BaseType::UNKNOWN;
    int tabIndex = -1; // indeks ke tab di symbol table
    int lev      = -1; // lexical level

    virtual ~ASTNode() = default;
    virtual void accept(ASTVisitor* visitor) = 0;
};
```

2. ASTVisitor

ASTVisitor digunakan untuk menelusuri AST secara Depth-First Search dan melakukan proses semantic analysis pada setiap jenis node. Setiap fungsi visit akan mengunjungi child node terlebih dahulu jika diperlukan, lalu melakukan aksi semantic seperti memasukkan deklarasi ke symbol table, melakukan lookup identifi er, menghitung tipe ekspresi, mengecek kompatibilitas tipe assignment, dan menambahkan anotasi pada node. Contohnya, saat mengunjungi VarDeclNode, visitor akan membaca tipe variabel lalu mendaftarkan setiap nama variabel ke symbol table. Saat mengunjungi VarNode, visitor akan melakukan lookup untuk memastikan identifi er sudah dideklarasikan. Hasil akhirnya adalah AST yang sudah terdekorasi dengan informasi tipe, indeks symbol table, dan lexical level.



```
ASTVisitor();

int getVisitedNodeCount();

SymbolTable& getSymbolTable();

// Struktur program / block
virtual void visit(ProgramNode* node);
virtual void visit(CompoundNode* node);

// Declaration nodes
virtual void visit(ConstDeclNode* node);
virtual void visit(TypeDeclNode* node);
virtual void visit(VarDeclNode* node);
virtual void visit(ParamNode* node);
virtual void visit(SubprogramDeclNode* node);

// Type nodes
virtual void visit(NamedTypeNode* node);
virtual void visit(ArrayTypeNode* node);
virtual void visit(RecordTypeNode* node);
virtual void visit(RangeNode* node);
virtual void visit(EnumNode* node);

// Statement nodes
virtual void visit(AssignNode* node);
virtual void visit(IfNode* node);
virtual void visit(CaseNode* node);
virtual void visit(CaseBlockNode* node);
virtual void visit(WhileNode* node);
virtual void visit(RepeatNode* node);
virtual void visit(ForNode* node);
virtual void visit(ProcCallNode* node);

// Variable / access nodes
virtual void visit(VarNode* node);
virtual void visit(ArrayAccessNode* node);
virtual void visit(RecordAccessNode* node);

// Expression / literal nodes
virtual void visit(BinOpNode* node);
virtual void visit(UnaryOpNode* node);
virtual void visit(NumberNode* node);
virtual void visit(StringNode* node);
virtual void visit(CharNode* node);
```



BAB III PENGUJIAN

Masukkan (Input)

```
program Hello;

var
  a, b: integer;

begin
  a := 5;
  b := a + 10;
  writeln('Result = ', b);
end.
```

Keluaran (output)

```
\-- <program>
|  \-- <program-header>
|  |  \-- PROGRAMSY
|  |  \-- IDENT(Hello)
|  |  \-- SEMICOLON
|  \-- <declaration-part>
|  |  \-- <var-declaration>
|  |  |  \-- VARSY
|  |  |  \-- <identifier-list>
|  |  |  |  \-- IDENT(a)
|  |  |  |  \-- COMMA
|  |  |  |  \-- IDENT(b)
|  |  |  \-- COLON
|  |  |  \-- <type>
|  |  |  |  \-- IDENT(integer)
|  |  |  \-- SEMICOLON
|  \-- <compound-statement>
|  |  \-- BEGINSY
|  |  \-- <statement-list>
|  |  |  \-- <statement>
|  |  |  |  \-- <assignment-statement>
|  |  |  |  |  \-- <variable>
|  |  |  |  |  |  \-- IDENT(a)
|  |  |  |  |  \-- BECOMES
|  |  |  |  |  \-- <expression>
|  |  |  |  |  |  \-- <simple-expression>
|  |  |  |  |  |  |  \-- <term>
|  |  |  |  |  |  |  |  \-- <factor>
|  |  |  |  |  |  |  |  \-- INTCON(5)
|  |  |  |  \-- SEMICOLON
|  |  |  \-- <statement>
|  |  |  |  \-- <assignment-statement>
|  |  |  |  |  \-- <variable>
|  |  |  |  |  |  \-- IDENT(b)
|  |  |  |  |  \-- BECOMES
|  |  |  |  |  \-- <expression>
|  |  |  |  |  |  \-- <simple-expression>
|  |  |  |  |  |  |  \-- <term>
|  |  |  |  |  |  |  |  \-- <factor>
|  |  |  |  |  |  |  |  |  \-- IDENT(a)
|  |  |  |  |  |  |  |  \-- <additive-operator>
```





2	BEGIN	keyword	void	null	1	0	0	-1
3	CASE	keyword	void	null	1	0	0	-1
4	CONST	keyword	void	null	1	0	0	-1
5	DIV	keyword	void	null	1	0	0	-1
6	DOWNT0	keyword	void	null	1	0	0	-1
7	DO	keyword	void	null	1	0	0	-1
8	ELSE	keyword	void	null	1	0	0	-1
9	END	keyword	void	null	1	0	0	-1
10	FOR	keyword	void	null	1	0	0	-1
11	FUNCTION	keyword	void	null	1	0	0	-1
12	IF	keyword	void	null	1	0	0	-1
13	MOD	keyword	void	null	1	0	0	-1
14	NOT	keyword	void	null	1	0	0	-1
15	OF	keyword	void	null	1	0	0	-1
16	OR	keyword	void	null	1	0	0	-1
17	PROCEDURE	keyword	void	null	1	0	0	-1
18	PROGRAM	keyword	void	null	1	0	0	-1
19	RECORD	keyword	void	null	1	0	0	-1
20	REPEAT	keyword	void	null	1	0	0	-1
21	INTEGER	type	integer	null	1	0	0	-1
22	REAL	type	real	null	1	0	0	-1
23	BOOLEAN	type	boolean	null	1	0	0	-1
24	CHAR	type	char	null	1	0	0	-1
25	STRING	type	string	null	1	0	0	-1
26	THEN	keyword	void	null	1	0	0	-1
27	TO	keyword	void	null	1	0	0	-1
28	TYPE	keyword	void	null	1	0	0	-1
29	UNTIL	keyword	void	null	1	0	0	-1
30	VAR	keyword	void	null	1	0	0	-1
31	WHILE	keyword	void	null	1	0	0	-1
32	TRUE	constant	boolean	null	1	0	0	-1
33	FALSE	constant	boolean	null	1	0	0	32
34	READ	procedure	void	null	1	0	0	33
35	READLN	procedure	void	null	1	0	0	34
36	WRITE	procedure	void	null	1	0	0	35
37	WRITELN	procedure	void	null	1	0	0	36
38	A	variable	integer	null	1	0	0	37
39	B	variable	integer	null	1	0	1	38

===== ATAB =====

idx	xtyp	etyp	eref	low	high	elsz	size
-----	------	------	------	-----	------	------	------

===== BTAB =====

idx	last	lpar	psze	vsze
-----	------	------	------	------

0	39	-1	0	2
---	----	----	---	---

Masukkan (Input)

```
program TestConstEnumRange;
```

```
const
  MAX == 100;
  MIN == -10;
  PI == 3.14;
  MSG == 'hello';
  CH == 'a';
```



```
type
  Direction == (north, south, east, west);
  Month == (jan, feb, mar, apr, may, jun);
  Letter == 'a'..'z';
  UpperLetter == 'A'..'Z';

var
  d: integer;
  i: integer;

begin
  d := 5;
  i := MAX;

  case d of
    0: writeln('nol');
    1, 2, 3: writeln('kecil');
    4, 5: writeln('sedang');
    6, 7, 8, 9: writeln('besar')
  end;

  case i of
    100: writeln('maksimum')
  end;
end.
```

Keluaran (output)

```
\-- <program>
|  \-- <program-header>
|  |  \-- PROGRAMSY
|  |  \-- IDENT (TestConstEnumRange)
|  |  \-- SEMICOLON
|  \-- <declaration-part>
|  |  \-- <const-declaration>
|  |  |  \-- CONSTSY
|  |  |  \-- IDENT (MAX)
|  |  |  \-- EQL
|  |  |  \-- <constant>
|  |  |  |  \-- INTCON (100)
|  |  |  \-- SEMICOLON
|  |  |  \-- IDENT (MIN)
|  |  |  \-- EQL
|  |  |  \-- <constant>
|  |  |  |  \-- INTCON (10)
|  |  |  \-- SEMICOLON
|  |  |  \-- IDENT (PI)
|  |  |  \-- EQL
|  |  |  \-- <constant>
|  |  |  |  \-- REALCON (3.14)
|  |  |  \-- SEMICOLON
|  |  |  \-- IDENT (MSG)
|  |  |  \-- EQL
|  |  |  \-- <constant>
|  |  |  |  \-- STRING ('hello')
|  |  |  \-- SEMICOLON
|  |  |  \-- IDENT (CH)
|  |  |  \-- EQL
|  |  |  \-- <constant>
|  |  |  |  \-- CHARCON ('a')
|  |  |  \-- SEMICOLON
|  |  \-- <type-declaration>
```



```
| | | -- TYPESY
| | | -- IDENT(Direction)
| | | -- EQL
| | | -- <type>
| | | | -- <enumerated>
| | | | | -- LPARENT
| | | | | -- IDENT(north)
| | | | | -- COMMA
| | | | | -- IDENT(south)
| | | | | -- COMMA
| | | | | -- IDENT(east)
| | | | | -- COMMA
| | | | | -- IDENT(west)
| | | | | -- RPARENT
| | | -- SEMICOLON
| | | -- IDENT(Month)
| | | -- EQL
| | | -- <type>
| | | | -- <enumerated>
| | | | | -- LPARENT
| | | | | -- IDENT(jan)
| | | | | -- COMMA
| | | | | -- IDENT(feb)
| | | | | -- COMMA
| | | | | -- IDENT(mar)
| | | | | -- COMMA
| | | | | -- IDENT(apr)
| | | | | -- COMMA
| | | | | -- IDENT(may)
| | | | | -- COMMA
| | | | | -- IDENT(jun)
| | | | | -- RPARENT
| | | -- SEMICOLON
| | | -- IDENT(Letter)
| | | -- EQL
| | | -- <type>
| | | | -- <range>
| | | | | -- <constant>
| | | | | | -- CHARCON('a')
| | | | | -- PERIOD
| | | | | -- PERIOD
| | | | | -- <constant>
| | | | | | -- CHARCON('z')
| | | -- SEMICOLON
| | | -- IDENT(UpperLetter)
| | | -- EQL
| | | -- <type>
| | | | -- <range>
| | | | | -- <constant>
| | | | | | -- CHARCON('A')
| | | | | -- PERIOD
| | | | | -- PERIOD
| | | | | -- <constant>
| | | | | | -- CHARCON('Z')
| | | | -- SEMICOLON
| | | -- <var-declaration>
| | | | -- VARSY
| | | | -- <identifier-list>
| | | | | -- IDENT(d)
| | | | -- COLON
| | | | -- <type>
| | | | | -- IDENT(integer)
| | | | -- SEMICOLON
| | | | -- <identifier-list>
| | | | | -- IDENT(i)
| | | | -- COLON
| | | | -- <type>
| | | | | -- IDENT(integer)
| | | | -- SEMICOLON
| | | -- <compound-statement>
```

```

|-- BEGINSY
|-- <statement-list>
|   |-- <statement>
|       |-- <assignment-statement>
|           |-- <variable>
|               |-- IDENT(d)
|           |-- BECOMES
|           |-- <expression>
|               |-- <simple-expression>
|                   |-- <term>
|                       |-- <factor>
|                           |-- INTCON(5)
|
|-- SEMICOLON
|-- <statement>
|   |-- <assignment-statement>
|       |-- <variable>
|           |-- IDENT(i)
|       |-- BECOMES
|       |-- <expression>
|           |-- <simple-expression>
|               |-- <term>
|                   |-- <factor>
|                       |-- IDENT(MAX)
|
|-- SEMICOLON
|-- <statement>
|   |-- <case-statement>
|       |-- CASESY
|       |-- <expression>
|           |-- <simple-expression>
|               |-- <term>
|                   |-- <factor>
|                       |-- IDENT(d)
|       |-- OFSY
|       |-- <case-block>
|           |-- <constant>
|               |-- INTCON(0)
|           |-- COLON
|           |-- <statement>
|               |-- <procedure/function-call>
|                   |-- IDENT(writeln)
|                   |-- LPARENT
|                   |-- <parameter-list>
|                       |-- <expression>
|                           |-- <simple-expression>
|                               |-- <term>
|                                   |-- <factor>
|                                       |-- STRING('nol')
|                   |-- RPARENT
|       |-- SEMICOLON
|       |-- <case-block>
|           |-- <constant>
|               |-- INTCON(1)
|           |-- COMMA
|           |-- <constant>
|               |-- INTCON(2)
|           |-- COMMA
|           |-- <constant>
|               |-- INTCON(3)
|           |-- COLON
|           |-- <statement>
|               |-- <procedure/function-call>
|                   |-- IDENT(writeln)
|                   |-- LPARENT
|                   |-- <parameter-list>
|                       |-- <expression>
|                           |-- <simple-expression>
|                               |-- <term>
|                                   |-- <factor>
|                                       |-- STRING('kecil')
|               |-- RPARENT

```

```

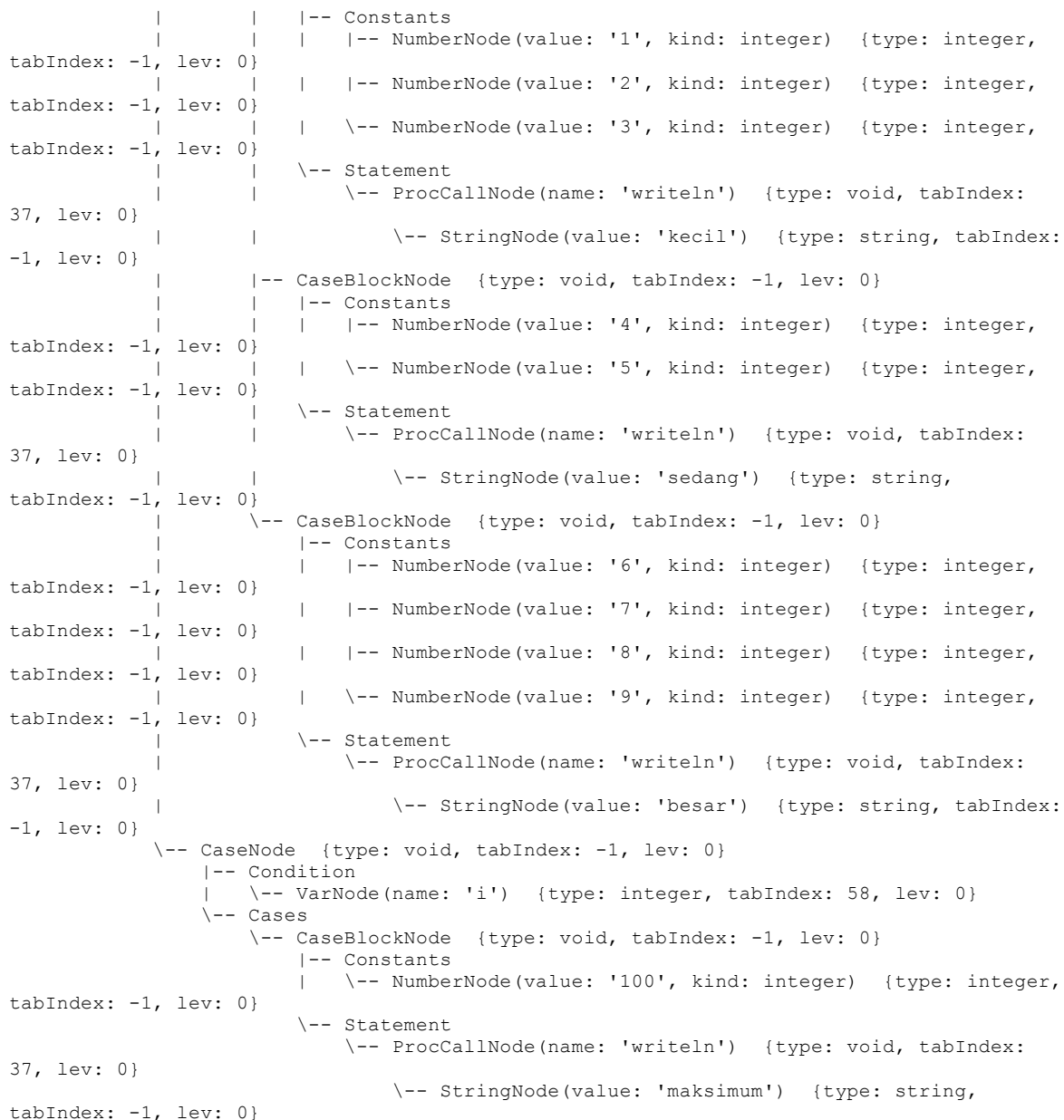
|-- SEMICOLON
\-- <case-block>
|-- <constant>
|   \-- INTCON(4)
|-- COMMA
|-- <constant>
|   \-- INTCON(5)
|-- COLON
|-- <statement>
|   \-- <procedure/function-call>
|       |-- IDENT(writeln)
|       |-- LPARENT
|       |-- <parameter-list>
|       |   \-- <expression>
|       |       \-- <simple-expression>
|       |           \-- <term>
|       |               \-- <factor>
|       |                   \-- STRING('sedang')
|       |-- RPARENT
|-- SEMICOLON
\-- <case-block>
|-- <constant>
|   \-- INTCON(6)
|-- COMMA
|-- <constant>
|   \-- INTCON(7)
|-- COMMA
|-- <constant>
|   \-- INTCON(8)
|-- COMMA
|-- <constant>
|   \-- INTCON(9)
|-- COLON
\-- <statement>
|   \-- <procedure/function-call>
|       |-- IDENT(writeln)
|       |-- LPARENT
|       |-- <parameter-list>
|       |   \-- <expression>
|       |       \-- <simple-expression>
|       |           \-- <term>
|       |               \-- <factor>
|       |                   \-- STRING('besar')
|       |-- RPARENT
|-- END SY
|-- SEMICOLON
\-- <statement>
\-- <case-statement>
|-- CASE SY
|-- <expression>
|   \-- <simple-expression>
|       \-- <term>
|           \-- <factor>
|               \-- IDENT(i)
|-- OF SY
\-- <case-block>
|-- <constant>
|   \-- INTCON(100)
|-- COLON
\-- <statement>
|   \-- <procedure/function-call>
|       |-- IDENT(writeln)
|       |-- LPARENT
|       |-- <parameter-list>
|       |   \-- <expression>
|       |       \-- <simple-expression>
|       |           \-- <term>
|       |               \-- <factor>
|       |                   \-- STRING('maksimum')
|       |-- RPARENT

```



```
| | | \-- ENDSY
| | \-- SEMICOLON
| \-- ENDSY
\-- PERIOD

===== DECORATED AST =====
\-- ProgramNode(name: 'TestConstEnumRange') {type: unknown, tabIndex: -1, lev: -1}
  |-- Declarations
  | | -- ConstDeclNode(name: 'MAX') {type: integer, tabIndex: 38, lev: 0}
  | | \-- NumberNode(value: '100', kind: integer) {type: integer, tabIndex: -1,
lev: 0}
  | | -- ConstDeclNode(name: 'MIN') {type: integer, tabIndex: 39, lev: 0}
  | | \-- NumberNode(value: '10', kind: integer) {type: integer, tabIndex: -1,
lev: 0}
  | | -- ConstDeclNode(name: 'PI') {type: real, tabIndex: 40, lev: 0}
  | | \-- NumberNode(value: '3.14', kind: real) {type: real, tabIndex: -1, lev:
0}
  | | -- ConstDeclNode(name: 'MSG') {type: string, tabIndex: 41, lev: 0}
  | | \-- StringNode(value: 'hello') {type: string, tabIndex: -1, lev: 0}
  | | -- ConstDeclNode(name: 'CH') {type: char, tabIndex: 42, lev: 0}
  | | \-- CharNode(value: 'a') {type: char, tabIndex: -1, lev: 0}
  | | -- TypeDeclNode(name: 'Direction') {type: enum, tabIndex: 47, lev: 0}
  | | \-- EnumNode(values: [north, south, east, west]) {type: enum, tabIndex: 46,
lev: 0}
  | | -- TypeDeclNode(name: 'Month') {type: enum, tabIndex: 54, lev: 0}
  | | \-- EnumNode(values: [jan, feb, mar, apr, may, jun]) {type: enum, tabIndex:
53, lev: 0}
  | | -- TypeDeclNode(name: 'Letter') {type: subrange, tabIndex: 55, lev: 0}
  | | \-- RangeNode {type: subrange, tabIndex: -1, lev: 0}
  | | | -- LowerBound
  | | | | \-- CharNode(value: 'a') {type: char, tabIndex: -1, lev: 0}
  | | | | -- UpperBound
  | | | | \-- CharNode(value: 'z') {type: char, tabIndex: -1, lev: 0}
  | | | -- TypeDeclNode(name: 'UpperLetter') {type: subrange, tabIndex: 56, lev: 0}
  | | | \-- RangeNode {type: subrange, tabIndex: -1, lev: 0}
  | | | | -- LowerBound
  | | | | | \-- CharNode(value: 'A') {type: char, tabIndex: -1, lev: 0}
  | | | | -- UpperBound
  | | | | | \-- CharNode(value: 'Z') {type: char, tabIndex: -1, lev: 0}
  | | | -- VarDeclNode(names: [d]) {type: integer, tabIndex: 57, lev: 0}
  | | | \-- NamedTypeNode(name: 'integer') {type: integer, tabIndex: -1, lev: 0}
  | | | -- VarDeclNode(names: [i]) {type: integer, tabIndex: 58, lev: 0}
  | | | \-- NamedTypeNode(name: 'integer') {type: integer, tabIndex: -1, lev: 0}
  | -- Block
  | | -- CompoundNode {type: unknown, tabIndex: -1, lev: 0}
  | | | -- AssignNode {type: void, tabIndex: -1, lev: 0}
  | | | | -- Target
  | | | | | \-- VarNode(name: 'd') {type: integer, tabIndex: 57, lev: 0}
  | | | | | -- Value
  | | | | | \-- NumberNode(value: '5', kind: integer) {type: integer, tabIndex:
-1, lev: 0}
  | | | | -- AssignNode {type: void, tabIndex: -1, lev: 0}
  | | | | | -- Target
  | | | | | | \-- VarNode(name: 'i') {type: integer, tabIndex: 58, lev: 0}
  | | | | | | -- Value
  | | | | | | \-- VarNode(name: 'MAX') {type: integer, tabIndex: 38, lev: 0}
  | | | | -- CaseNode {type: void, tabIndex: -1, lev: 0}
  | | | | | -- Condition
  | | | | | | \-- VarNode(name: 'd') {type: integer, tabIndex: 57, lev: 0}
  | | | | | -- Cases
  | | | | | | -- CaseBlockNode {type: void, tabIndex: -1, lev: 0}
  | | | | | | | -- Constants
  | | | | | | | | \-- NumberNode(value: '0', kind: integer) {type: integer,
tabIndex: -1, lev: 0}
  | | | | | | | -- Statement
  | | | | | | | | -- ProcCallNode(name: 'writeln') {type: void, tabIndex:
37, lev: 0}
  | | | | | | | | \-- StringNode(value: 'nol') {type: string, tabIndex:
-1, lev: 0}
  | | | | | | | -- CaseBlockNode {type: void, tabIndex: -1, lev: 0}
```



===== TAB =====

idx	identifier	obj	type	ref	nrm	lev	adr	link
0	AND	keyword	void	null	1	0	0	-1
1	ARRAY	keyword	void	null	1	0	0	-1
2	BEGIN	keyword	void	null	1	0	0	-1
3	CASE	keyword	void	null	1	0	0	-1
4	CONST	keyword	void	null	1	0	0	-1
5	DIV	keyword	void	null	1	0	0	-1
6	DOWNT0	keyword	void	null	1	0	0	-1
7	DO	keyword	void	null	1	0	0	-1
8	ELSE	keyword	void	null	1	0	0	-1
9	END	keyword	void	null	1	0	0	-1
10	FOR	keyword	void	null	1	0	0	-1
11	FUNCTION	keyword	void	null	1	0	0	-1
12	IF	keyword	void	null	1	0	0	-1
13	MOD	keyword	void	null	1	0	0	-1



14	NOT	keyword	void	null	1	0	0	-1
15	OF	keyword	void	null	1	0	0	-1
16	OR	keyword	void	null	1	0	0	-1
17	PROCEDURE	keyword	void	null	1	0	0	-1
18	PROGRAM	keyword	void	null	1	0	0	-1
19	RECORD	keyword	void	null	1	0	0	-1
20	REPEAT	keyword	void	null	1	0	0	-1
21	INTEGER	type	integer	null	1	0	0	-1
22	REAL	type	real	null	1	0	0	-1
23	BOOLEAN	type	boolean	null	1	0	0	-1
24	CHAR	type	char	null	1	0	0	-1
25	STRING	type	string	null	1	0	0	-1
26	THEN	keyword	void	null	1	0	0	-1
27	TO	keyword	void	null	1	0	0	-1
28	TYPE	keyword	void	null	1	0	0	-1
29	UNTIL	keyword	void	null	1	0	0	-1
30	VAR	keyword	void	null	1	0	0	-1
31	WHILE	keyword	void	null	1	0	0	-1
32	TRUE	constant	boolean	null	1	0	0	-1
33	FALSE	constant	boolean	null	1	0	0	32
34	READ	procedure	void	null	1	0	0	33
35	READLN	procedure	void	null	1	0	0	34
36	WRITE	procedure	void	null	1	0	0	35
37	WRITELN	procedure	void	null	1	0	0	36
38	MAX	constant	integer	null	1	0	0	37
39	MIN	constant	integer	null	1	0	0	38
40	PI	constant	real	null	1	0	0	39
41	MSG	constant	string	null	1	0	0	40
42	CH	constant	char	null	1	0	0	41
43	NORTH	constant	enum	null	1	0	0	42
44	SOUTH	constant	enum	null	1	0	0	43
45	EAST	constant	enum	null	1	0	0	44
46	WEST	constant	enum	null	1	0	0	45
47	DIRECTION	type	enum	null	1	0	0	46
48	JAN	constant	enum	null	1	0	0	47
49	FEB	constant	enum	null	1	0	0	48
50	MAR	constant	enum	null	1	0	0	49
51	APR	constant	enum	null	1	0	0	50
52	MAY	constant	enum	null	1	0	0	51
53	JUN	constant	enum	null	1	0	0	52
54	MONTH	type	enum	null	1	0	0	53
55	LETTER	type	subrange	null	1	0	0	54
56	UPPERLETTER	type	subrange	null	1	0	0	55
57	D	variable	integer	null	1	0	0	56
58	I	variable	integer	null	1	0	1	57

===== ATAB =====

idx	xtype	etyp	eref	low	high	elsz	size
-----	-------	------	------	-----	------	------	------

===== BTAB =====

idx	last	lpar	psze	vsze
-----	------	------	------	------

0	58	-1	0	2
---	----	----	---	---

Masukkan (Input)

```
program TestRecordArray;
```

```
type
```

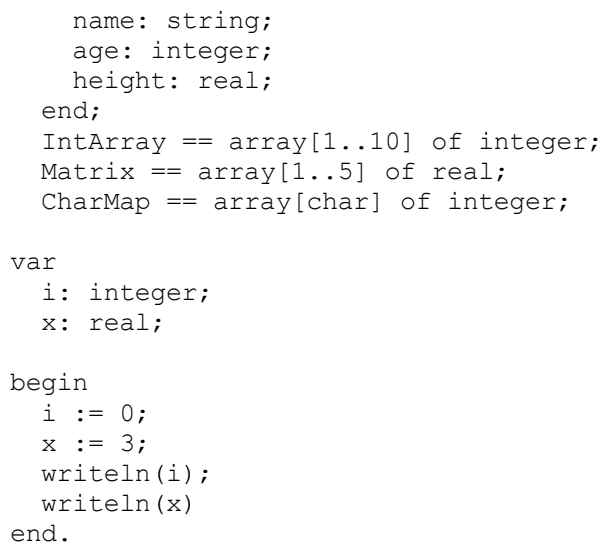
```
  Point == record
```

```
    x, y: integer;
```

```
    label: string;
```

```
end;
```

```
  Person == record
```



```

\-- <program>
|  \-- <program-header>
|  |  \-- PROGRAMSY
|  |  \-- IDENT (TestRecordArray)
|  |  \-- SEMICOLON
|  \-- <declaration-part>
|  |  \-- <type-declaration>
|  |  |  \-- TYPESY
|  |  |  \-- IDENT (Point)
|  |  |  \-- EQL
|  |  |  \-- <type>
|  |  |  |  \-- <record-type>
|  |  |  |  |  \-- RECORDSY
|  |  |  |  |  \-- <field-list>
|  |  |  |  |  |  \-- <field-part>
|  |  |  |  |  |  |  \-- <identifier-list>
|  |  |  |  |  |  |  |  \-- IDENT (x)
|  |  |  |  |  |  |  |  \-- COMMA
|  |  |  |  |  |  |  |  \-- IDENT (y)
|  |  |  |  |  |  |  \-- COLON
|  |  |  |  |  |  \-- <type>
|  |  |  |  |  |  |  \-- IDENT (integer)
|  |  |  |  |  \-- SEMICOLON
|  |  |  |  \-- <field-part>
|  |  |  |  |  \-- <identifier-list>
|  |  |  |  |  |  \-- IDENT (label)
|  |  |  |  |  \-- COLON
|  |  |  |  |  \-- <type>
|  |  |  |  |  |  \-- IDENT (string)
|  |  |  |  \-- SEMICOLON
|  |  |  \-- ENDSY
|  |  \-- SEMICOLON
|  |  \-- IDENT (Person)
|  |  \-- EQL
|  |  \-- <type>
|  |  |  \-- <record-type>
|  |  |  |  \-- RECORDSY
|  |  |  |  \-- <field-list>
|  |  |  |  |  \-- <field-part>
|  |  |  |  |  |  \-- <identifier-list>
|  |  |  |  |  |  |  \-- IDENT (name)
|  |  |  |  |  |  \-- COLON
|  |  |  |  |  \-- <type>

```




```
|      |-- COLON
|      |-- <type>
|      |   \-- IDENT(integer)
|      |-- SEMICOLON
|      |-- <identifier-list>
|      |   \-- IDENT(x)
|      |-- COLON
|      |-- <type>
|      |   \-- IDENT(real)
|      |-- SEMICOLON
|-- <compound-statement>
|-- BEGINSY
|-- <statement-list>
|   |-- <statement>
|   |   \-- <assignment-statement>
|   |       |-- <variable>
|   |       |   \-- IDENT(i)
|   |       |-- BECOMES
|   |       \-- <expression>
|   |           \-- <simple-expression>
|   |               \-- <term>
|   |                   \-- <factor>
|   |                       \-- INTCON(0)
|   |-- SEMICOLON
|   |-- <statement>
|   |   \-- <assignment-statement>
|   |       |-- <variable>
|   |       |   \-- IDENT(x)
|   |       |-- BECOMES
|   |       \-- <expression>
|   |           \-- <simple-expression>
|   |               \-- <term>
|   |                   \-- <factor>
|   |                       \-- INTCON(3)
|   |-- SEMICOLON
|   |-- <statement>
|   |   \-- <procedure/function-call>
|   |       |-- IDENT(writeln)
|   |       |-- LPARENT
|   |       |-- <parameter-list>
|   |       |   \-- <expression>
|   |       |       \-- <simple-expression>
|   |       |           \-- <term>
|   |       |               \-- <factor>
|   |       |                   \-- IDENT(i)
|   |       |-- RPARENT
|   |-- SEMICOLON
|   |-- <statement>
|   |   \-- <procedure/function-call>
|   |       |-- IDENT(writeln)
|   |       |-- LPARENT
|   |       |-- <parameter-list>
|   |       |   \-- <expression>
|   |       |       \-- <simple-expression>
|   |       |           \-- <term>
|   |       |               \-- <factor>
|   |       |                   \-- IDENT(x)
|   |       |-- RPARENT
|   |-- ENDSY
|-- PERIOD

===== DECORATED AST =====
|-- ProgramNode(name: 'TestRecordArray') {type: unknown, tabIndex: -1, lev: -1}
|   |-- Declarations
|   |   |-- TypeDeclNode(name: 'Point') {type: record, tabIndex: 41, lev: 0}
|   |   |   \-- RecordTypeNode {type: record, tabIndex: 2, lev: 0}
|   |   |       |-- VarDeclNode(names: [x, y]) {type: integer, tabIndex: 39, lev: 1}
|   |   |       |   \-- NamedTypeNode(name: 'integer') {type: integer, tabIndex: -1,
|   |   |       |       lev: 1}
|   |   |       \-- VarDeclNode(names: [label]) {type: string, tabIndex: 40, lev: 1}
```



```
| | | \-- NamedTypeNode(name: 'string') {type: string, tabIndex: -1, lev:
1}
| | | \-- TypeDeclNode(name: 'Person') {type: record, tabIndex: 45, lev: 0}
| | | \-- RecordTypeNode {type: record, tabIndex: 4, lev: 0}
| | | | \-- VarDeclNode(names: [name]) {type: string, tabIndex: 42, lev: 1}
| | | | | \-- NamedTypeNode(name: 'string') {type: string, tabIndex: -1, lev:
1}
| | | | | \-- VarDeclNode(names: [age]) {type: integer, tabIndex: 43, lev: 1}
| | | | | | \-- NamedTypeNode(name: 'integer') {type: integer, tabIndex: -1,
lev: 1}
| | | | | | \-- VarDeclNode(names: [height]) {type: real, tabIndex: 44, lev: 1}
| | | | | | | \-- NamedTypeNode(name: 'real') {type: real, tabIndex: -1, lev: 1}
| | | | | \-- TypeDeclNode(name: 'IntArray') {type: array, tabIndex: 46, lev: 0}
| | | | | \-- ArrayTypeNode {type: array, tabIndex: 0, lev: 0}
| | | | | | \-- IndexType
| | | | | | | \-- RangeNode {type: unknown, tabIndex: -1, lev: -1}
| | | | | | | | \-- LowerBound
| | | | | | | | | \-- NumberNode(value: '1', kind: integer) {type: unknown,
tabIndex: -1, lev: -1}
| | | | | | | | | \-- UpperBound
| | | | | | | | | | \-- NumberNode(value: '10', kind: integer) {type: unknown,
tabIndex: -1, lev: -1}
| | | | | | | | | \-- ElementType
| | | | | | | | | \-- NamedTypeNode(name: 'integer') {type: unknown, tabIndex: -1,
lev: -1}
| | | | | \-- TypeDeclNode(name: 'Matrix') {type: array, tabIndex: 47, lev: 0}
| | | | | \-- ArrayTypeNode {type: array, tabIndex: 1, lev: 0}
| | | | | | \-- IndexType
| | | | | | | \-- RangeNode {type: unknown, tabIndex: -1, lev: -1}
| | | | | | | | \-- LowerBound
| | | | | | | | | \-- NumberNode(value: '1', kind: integer) {type: unknown,
tabIndex: -1, lev: -1}
| | | | | | | | | \-- UpperBound
| | | | | | | | | | \-- NumberNode(value: '5', kind: integer) {type: unknown,
tabIndex: -1, lev: -1}
| | | | | | | | | \-- ElementType
| | | | | | | | | \-- NamedTypeNode(name: 'real') {type: unknown, tabIndex: -1, lev:
-1}
| | | | | \-- TypeDeclNode(name: 'CharMap') {type: array, tabIndex: 48, lev: 0}
| | | | | \-- ArrayTypeNode {type: array, tabIndex: 2, lev: 0}
| | | | | | \-- IndexType
| | | | | | | \-- NamedTypeNode(name: 'char') {type: unknown, tabIndex: -1, lev:
-1}
| | | | | | | \-- ElementType
| | | | | | | | \-- NamedTypeNode(name: 'integer') {type: unknown, tabIndex: -1,
lev: -1}
| | | | | \-- VarDeclNode(names: [i]) {type: integer, tabIndex: 49, lev: 0}
| | | | | | \-- NamedTypeNode(name: 'integer') {type: integer, tabIndex: -1, lev: 0}
| | | | | \-- VarDeclNode(names: [x]) {type: real, tabIndex: 50, lev: 0}
| | | | | | \-- NamedTypeNode(name: 'real') {type: real, tabIndex: -1, lev: 0}
| | | | | \-- Block
| | | | | | \-- CompoundNode {type: unknown, tabIndex: -1, lev: 0}
| | | | | | | \-- AssignNode {type: void, tabIndex: -1, lev: 0}
| | | | | | | | \-- Target
| | | | | | | | | \-- VarNode(name: 'i') {type: integer, tabIndex: 49, lev: 0}
| | | | | | | | | \-- Value
| | | | | | | | | | \-- NumberNode(value: '0', kind: integer) {type: integer, tabIndex:
-1, lev: 0}
| | | | | | | | | \-- AssignNode {type: void, tabIndex: -1, lev: 0}
| | | | | | | | | | \-- Target
| | | | | | | | | | | \-- VarNode(name: 'x') {type: real, tabIndex: 50, lev: 0}
| | | | | | | | | | | \-- Value
| | | | | | | | | | | | \-- NumberNode(value: '3', kind: integer) {type: integer, tabIndex:
-1, lev: 0}
| | | | | | \-- ProcCallNode(name: 'writeln') {type: void, tabIndex: 37, lev: 0}
| | | | | | | \-- VarNode(name: 'i') {type: integer, tabIndex: 49, lev: 0}
| | | | | | \-- ProcCallNode(name: 'writeln') {type: void, tabIndex: 37, lev: 0}
| | | | | | | \-- VarNode(name: 'x') {type: real, tabIndex: 50, lev: 0}

===== SYMBOL TABLE =====
```



===== TAB =====

idx	identifier	obj	type	ref	nrm	lev	adr	link
0	AND	keyword	void	null	1	0	0	-1
1	ARRAY	keyword	void	null	1	0	0	-1
2	BEGIN	keyword	void	null	1	0	0	-1
3	CASE	keyword	void	null	1	0	0	-1
4	CONST	keyword	void	null	1	0	0	-1
5	DIV	keyword	void	null	1	0	0	-1
6	DOWNT0	keyword	void	null	1	0	0	-1
7	DO	keyword	void	null	1	0	0	-1
8	ELSE	keyword	void	null	1	0	0	-1
9	END	keyword	void	null	1	0	0	-1
10	FOR	keyword	void	null	1	0	0	-1
11	FUNCTION	keyword	void	null	1	0	0	-1
12	IF	keyword	void	null	1	0	0	-1
13	MOD	keyword	void	null	1	0	0	-1
14	NOT	keyword	void	null	1	0	0	-1
15	OF	keyword	void	null	1	0	0	-1
16	OR	keyword	void	null	1	0	0	-1
17	PROCEDURE	keyword	void	null	1	0	0	-1
18	PROGRAM	keyword	void	null	1	0	0	-1
19	RECORD	keyword	void	null	1	0	0	-1
20	REPEAT	keyword	void	null	1	0	0	-1
21	INTEGER	type	integer	null	1	0	0	-1
22	REAL	type	real	null	1	0	0	-1
23	BOOLEAN	type	boolean	null	1	0	0	-1
24	CHAR	type	char	null	1	0	0	-1
25	STRING	type	string	null	1	0	0	-1
26	THEN	keyword	void	null	1	0	0	-1
27	TO	keyword	void	null	1	0	0	-1
28	TYPE	keyword	void	null	1	0	0	-1
29	UNTIL	keyword	void	null	1	0	0	-1
30	VAR	keyword	void	null	1	0	0	-1
31	WHILE	keyword	void	null	1	0	0	-1
32	TRUE	constant	boolean	null	1	0	0	-1
33	FALSE	constant	boolean	null	1	0	0	32
34	READ	procedure	void	null	1	0	0	33
35	READLN	procedure	void	null	1	0	0	34
36	WRITE	procedure	void	null	1	0	0	35
37	WRITELN	procedure	void	null	1	0	0	36
38	X	variable	integer	null	1	1	0	-1
39	Y	variable	integer	null	1	1	1	38
40	LABEL	variable	string	null	1	1	2	39
41	POINT	type	record	null	1	0	0	37
42	NAME	variable	string	null	1	1	0	-1
43	AGE	variable	integer	null	1	1	1	42
44	HEIGHT	variable	real	null	1	1	2	43
45	PERSON	type	record	null	1	0	0	41
46	INTARRAY	type	array	null	1	0	0	45
47	MATRIX	type	array	null	1	0	0	46
48	CHARMAP	type	array	null	1	0	0	47
49	I	variable	integer	null	1	0	0	48
50	X	variable	real	null	1	0	1	49

===== ATAB =====

idx	xtype	etyp	eref	low	high	elsz	size
0	unknown	unknown	null	0	0	0	0
1	unknown	unknown	null	0	0	0	0
2	unknown	unknown	null	0	0	0	0

===== BTAB =====

idx	last	lpar	psze	vsze
0	37	-1	0	0
1	40	-1	0	3
2	41	-1	0	0
3	44	-1	0	3
4	50	-1	0	2



Masukkan (Input)

```
program TestControlFlow;

var
  a, b, c: integer;
  x: real;

begin
  a := 10;
  b := 3;
  x := 3.14;

  c := a + b * 2 - 1;
  x := x / 2.0;

  if a > b then
    writeln('a lebih besar')
  else
    writeln('b lebih besar atau sama');

  if (a == 10) and (b <> 0) then
    writeln('valid')
  else if (a < 0) or (b <= 0) then
    writeln('negatif');

  while a > 0 do
    a := a - 1;

  repeat
    b := b + 1
  until b >= 10;

  for c := 1 to 5 do
    writeln(c);

  for c := 5 downto 1 do
    writeln(c);

  writeln('selesai', c, x)
end.
```

Keluaran (output)

```
\-- <program>
|  \-- <program-header>
|  |  \-- PROGRAMSY
|  |  \-- IDENT(TestControlFlow)
|  |  \-- SEMICOLON
|  \-- <declaration-part>
|  |  \-- <var-declaration>
|  |  \-- VARSY
```



```
|      |-- <identifier-list>
|      |      |-- IDENT(a)
|      |      |-- COMMA
|      |      |-- IDENT(b)
|      |      |-- COMMA
|      |      |-- IDENT(c)
|      |-- COLON
|      |-- <type>
|      |      |-- IDENT(integer)
|      |-- SEMICOLON
|      |-- <identifier-list>
|      |      |-- IDENT(x)
|      |-- COLON
|      |-- <type>
|      |      |-- IDENT(real)
|      |-- SEMICOLON
|-- <compound-statement>
|  |-- BEGINSY
|  |-- <statement-list>
|  |  |-- <statement>
|  |  |  |-- <assignment-statement>
|  |  |  |  |-- <variable>
|  |  |  |  |  |-- IDENT(a)
|  |  |  |  |-- BECOMES
|  |  |  |-- <expression>
|  |  |  |  |-- <simple-expression>
|  |  |  |  |  |-- <term>
|  |  |  |  |  |  |-- <factor>
|  |  |  |  |  |  |-- INTCON(10)
|  |  |-- SEMICOLON
|  |  |-- <statement>
|  |  |  |-- <assignment-statement>
|  |  |  |  |-- <variable>
|  |  |  |  |  |-- IDENT(b)
|  |  |  |  |-- BECOMES
|  |  |  |-- <expression>
|  |  |  |  |-- <simple-expression>
|  |  |  |  |  |-- <term>
|  |  |  |  |  |  |-- <factor>
|  |  |  |  |  |  |-- INTCON(3)
|  |  |-- SEMICOLON
|  |  |-- <statement>
|  |  |  |-- <assignment-statement>
|  |  |  |  |-- <variable>
|  |  |  |  |  |-- IDENT(x)
|  |  |  |  |-- BECOMES
|  |  |  |-- <expression>
|  |  |  |  |-- <simple-expression>
|  |  |  |  |  |-- <term>
|  |  |  |  |  |  |-- <factor>
|  |  |  |  |  |  |-- REALCON(3.14)
|  |  |-- SEMICOLON
|  |  |-- <statement>
|  |  |  |-- <assignment-statement>
|  |  |  |  |-- <variable>
|  |  |  |  |  |-- IDENT(c)
|  |  |  |  |-- BECOMES
|  |  |  |-- <expression>
|  |  |  |  |-- <simple-expression>
|  |  |  |  |  |-- <term>
|  |  |  |  |  |  |-- <factor>
|  |  |  |  |  |  |  |-- IDENT(a)
|  |  |  |  |  |-- <additive-operator>
|  |  |  |  |  |  |-- PLUS
|  |  |  |  |-- <term>
|  |  |  |  |  |-- <factor>
|  |  |  |  |  |  |-- IDENT(b)
|  |  |  |  |-- <multiplicative-operator>
|  |  |  |  |  |-- TIMES
|  |  |  |  |-- <factor>
```

```

|         |-- INTCON(2)
|         |-- <additive-operator>
|         |-- MINUS
|         |-- <term>
|         |-- <factor>
|         |-- INTCON(1)
|-- SEMICOLON
|-- <statement>
|   |-- <assignment-statement>
|   |   |-- <variable>
|   |   |-- IDENT(x)
|   |   |-- BECOMES
|   |   |-- <expression>
|   |   |   |-- <simple-expression>
|   |   |   |   |-- <term>
|   |   |   |   |   |-- <factor>
|   |   |   |   |   |-- IDENT(x)
|   |   |   |   |   |-- <multiplicative-operator>
|   |   |   |   |   |-- RDIV
|   |   |   |   |   |-- <factor>
|   |   |   |   |   |-- REALCON(2.0)
|-- SEMICOLON
|-- <statement>
|   |-- <if-statement>
|   |   |-- IFSY
|   |   |-- <expression>
|   |   |   |-- <simple-expression>
|   |   |   |   |-- <term>
|   |   |   |   |   |-- <factor>
|   |   |   |   |   |-- IDENT(a)
|   |   |   |-- <relational-operator>
|   |   |   |   |-- GTR
|   |   |   |-- <simple-expression>
|   |   |   |   |-- <term>
|   |   |   |   |   |-- <factor>
|   |   |   |   |   |-- IDENT(b)
|   |   |-- THENSY
|   |   |-- <statement>
|   |   |   |-- <procedure/function-call>
|   |   |   |   |-- IDENT(writeln)
|   |   |   |   |-- LPARENT
|   |   |   |   |-- <parameter-list>
|   |   |   |   |   |-- <expression>
|   |   |   |   |   |   |-- <simple-expression>
|   |   |   |   |   |   |   |-- <term>
|   |   |   |   |   |   |   |   |-- <factor>
|   |   |   |   |   |   |   |   |-- STRING('a lebih besar')
|   |   |   |   |-- RPARENT
|   |   |-- ELSESY
|   |   |-- <statement>
|   |   |   |-- <procedure/function-call>
|   |   |   |   |-- IDENT(writeln)
|   |   |   |   |-- LPARENT
|   |   |   |   |-- <parameter-list>
|   |   |   |   |   |-- <expression>
|   |   |   |   |   |   |-- <simple-expression>
|   |   |   |   |   |   |   |-- <term>
|   |   |   |   |   |   |   |   |-- <factor>
|   |   |   |   |   |   |   |   |-- STRING('b lebih besar atau sama')
|   |   |   |   |-- RPARENT
|-- SEMICOLON
|-- <statement>
|   |-- <if-statement>
|   |   |-- IFSY
|   |   |-- <expression>
|   |   |   |-- <simple-expression>
|   |   |   |   |-- <term>
|   |   |   |   |   |-- <factor>
|   |   |   |   |   |-- LPARENT
|   |   |   |   |   |-- <expression>

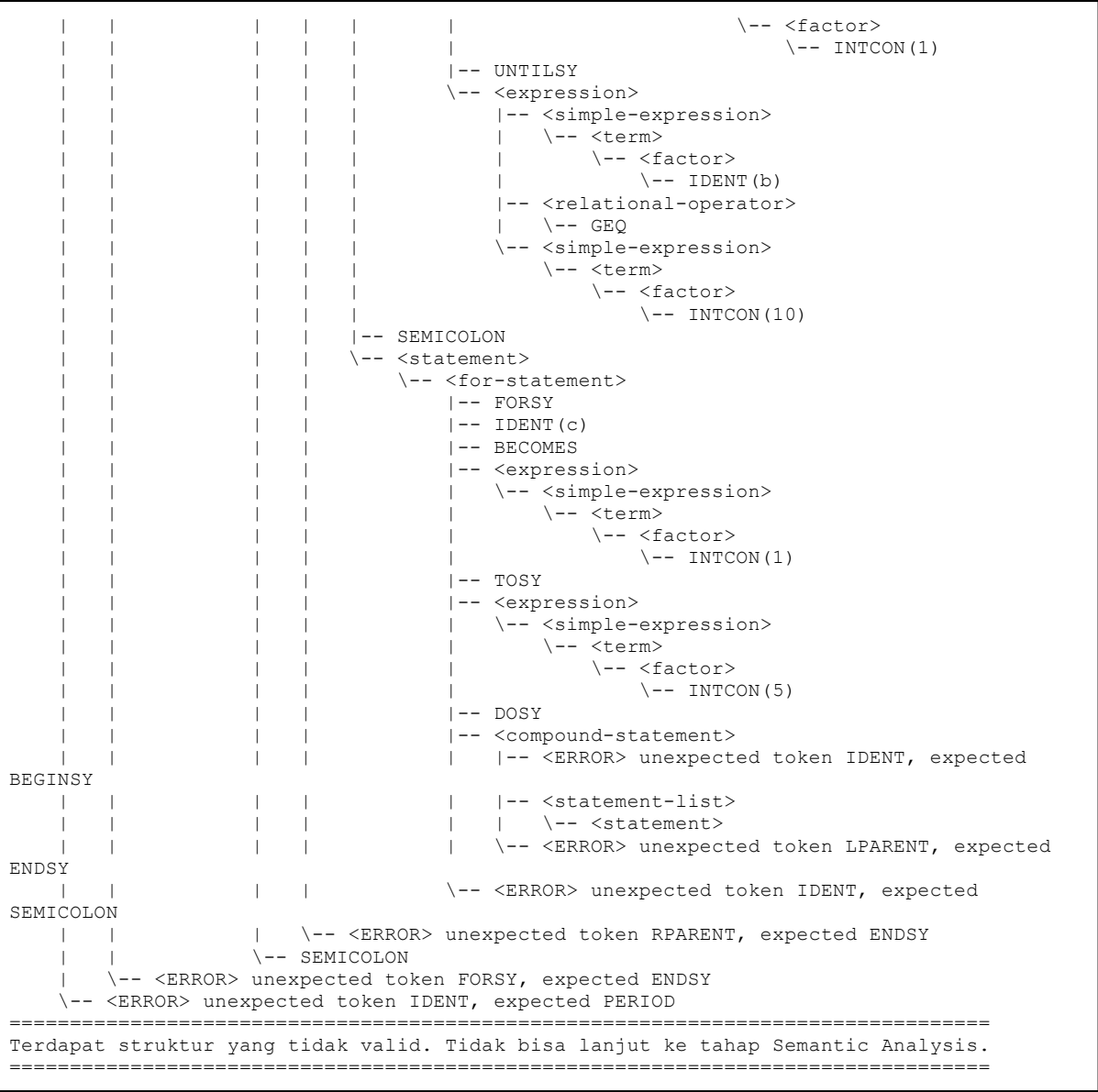
```



```

|-- <relational-operator>
| \-- LEQ
| \-- <simple-expression>
|   \-- <term>
|     \-- <factor>
|       \-- INTCON(0)
| \-- RPARENT
|-- THENSY
\-- <statement>
  \-- <procedure/function-call>
    |-- IDENT(writeln)
    |-- LPARENT
    |-- <parameter-list>
    | \-- <expression>
    |   \-- <simple-expression>
    |     \-- <term>
    |       \-- <factor>
    |         \-- STRING('negatif')
    | \-- RPARENT
  \-- SEMICOLON
\-- <statement>
  \-- <while-statement>
    |-- WHILESY
    |-- <expression>
    | \-- <simple-expression>
    |   \-- <term>
    |     \-- <factor>
    |       \-- IDENT(a)
    | \-- <relational-operator>
    |   \-- GTR
    | \-- <simple-expression>
    |   \-- <term>
    |     \-- <factor>
    |       \-- INTCON(0)
    \-- <compound-statement>
      |-- <ERROR> unexpected token DOSY, expected BEGINSY
      |-- <statement-list>
      | \-- <statement>
      |   \-- <assignment-statement>
      |     |-- <variable>
      |       \-- IDENT(a)
      |     |-- BECOMES
      |     \-- <expression>
      |       \-- <simple-expression>
      |         \-- <term>
      |           \-- <factor>
      |             \-- IDENT(a)
      |           \-- <additive-operator>
      |             \-- MINUS
      |           \-- <term>
      |             \-- <factor>
      |               \-- INTCON(1)
      \-- SEMICOLON
    \-- <statement>
      \-- <repeat-statement>
        |-- REPEATSY
        |-- <statement-list>
        | \-- <statement>
        |   \-- <assignment-statement>
        |     |-- <variable>
        |       \-- IDENT(b)
        |     |-- BECOMES
        |     \-- <expression>
        |       \-- <simple-expression>
        |         \-- <term>
        |           \-- <factor>
        |             \-- IDENT(b)
        |           \-- <additive-operator>
        |             \-- PLUS
        |           \-- <term>

```



```
program TestTiga;

type
    DataSiswa == record
        nomorInduk: integer;
        kodeKelas: char;
        nilaiAkhir: real;
    end;

var
    siswa: DataSiswa;
    i, batas, akumulasi: integer;
```



```
begin
  batas := 10;
  akumulasi := 0;

  for i := 1 to batas do
  begin
    akumulasi := akumulasi + i
  end;

  while akumulasi < 100 do
  begin
    akumulasi := akumulasi + 15;
    batas := batas - 1;
  end;
end.
```

Keluaran (output)

```
\-- <program>
|  \-- <program-header>
|  |  \-- PROGRAMSY
|  |  \-- IDENT(TestTiga)
|  |  \-- SEMICOLON
|  \-- <declaration-part>
|  |  \-- <type-declaration>
|  |  |  \-- TYPESY
|  |  |  |  \-- IDENT(DataSiswa)
|  |  |  |  \-- EQL
|  |  |  |  \-- <type>
|  |  |  |  |  \-- <record-type>
|  |  |  |  |  |  \-- RECORDSY
|  |  |  |  |  |  |  \-- <field-list>
|  |  |  |  |  |  |  |  \-- <field-part>
|  |  |  |  |  |  |  |  |  \-- <identifier-list>
|  |  |  |  |  |  |  |  |  |  \-- IDENT(nomorInduk)
|  |  |  |  |  |  |  |  |  |  \-- COLON
|  |  |  |  |  |  |  |  |  |  \-- <type>
|  |  |  |  |  |  |  |  |  |  |  \-- IDENT(integer)
|  |  |  |  |  |  |  |  |  |  \-- SEMICOLON
|  |  |  |  |  |  |  |  |  \-- <field-part>
|  |  |  |  |  |  |  |  |  |  \-- <identifier-list>
|  |  |  |  |  |  |  |  |  |  |  \-- IDENT(kodeKelas)
|  |  |  |  |  |  |  |  |  |  \-- COLON
|  |  |  |  |  |  |  |  |  |  \-- <type>
|  |  |  |  |  |  |  |  |  |  |  \-- IDENT(char)
|  |  |  |  |  |  |  |  |  |  \-- SEMICOLON
|  |  |  |  |  |  |  |  |  \-- <field-part>
|  |  |  |  |  |  |  |  |  |  \-- <identifier-list>
|  |  |  |  |  |  |  |  |  |  |  \-- IDENT(nilaiAkhir)
|  |  |  |  |  |  |  |  |  |  \-- COLON
|  |  |  |  |  |  |  |  |  |  \-- <type>
|  |  |  |  |  |  |  |  |  |  |  \-- IDENT(real)
|  |  |  |  |  |  |  |  |  |  \-- SEMICOLON
|  |  |  |  |  |  |  |  |  \-- ENDSY
|  |  |  |  \-- SEMICOLON
|  \-- <var-declaration>
|  |  \-- VARSY
|  |  |  \-- <identifier-list>
|  |  |  |  \-- IDENT(siswa)
|  |  |  |  \-- COLON
|  |  |  |  \-- <type>
|  |  |  |  |  \-- IDENT(DataSiswa)
|  |  |  |  \-- SEMICOLON
|  |  |  \-- <identifier-list>
|  |  |  |  \-- IDENT(i)
```

```

|-- COMMA
|-- IDENT(batas)
|-- COMMA
|-- IDENT(akumulasi)
|-- COLON
|-- <type>
|-- IDENT(integer)
|-- SEMICOLON
|-- <compound-statement>
|-- BEGINSY
|-- <statement-list>
|-- <statement>
|-- <assignment-statement>
|-- <variable>
|-- IDENT(batas)
|-- BECOMES
|-- <expression>
|-- <simple-expression>
|-- <term>
|-- <factor>
|-- INTCON(10)
|-- SEMICOLON
|-- <statement>
|-- <assignment-statement>
|-- <variable>
|-- IDENT(akumulasi)
|-- BECOMES
|-- <expression>
|-- <simple-expression>
|-- <term>
|-- <factor>
|-- INTCON(0)
|-- SEMICOLON
|-- <statement>
|-- <for-statement>
|-- FORSY
|-- IDENT(i)
|-- BECOMES
|-- <expression>
|-- <simple-expression>
|-- <term>
|-- <factor>
|-- INTCON(1)
|-- TOSY
|-- <expression>
|-- <simple-expression>
|-- <term>
|-- <factor>
|-- IDENT(batas)
|-- DOSY
|-- <compound-statement>
|-- BEGINSY
|-- <statement-list>
|-- <statement>
|-- <assignment-statement>
|-- <variable>
|-- IDENT(akumulasi)
|-- BECOMES
|-- <expression>
|-- <simple-expression>
|-- <term>
|-- <factor>
|-- IDENT(akumulasi)
|-- <additive-operator>
|-- PLUS
|-- <term>
|-- <factor>
|-- IDENT(i)
|-- ENDSY
|-- SEMICOLON

```

```

|-- <statement>
|-- <while-statement>
|   |-- WHILESY
|   |-- <expression>
|   |   |-- <simple-expression>
|   |   |   |-- <term>
|   |   |   |   |-- <factor>
|   |   |   |   |   |-- IDENT(akumulasi)
|   |   |-- <relational-operator>
|   |   |   |-- LSS
|   |-- <simple-expression>
|   |   |-- <term>
|   |   |   |-- <factor>
|   |   |   |   |-- INTCON(100)
|-- DOSY
|-- <compound-statement>
|   |-- BEGINSY
|   |-- <statement-list>
|   |   |-- <statement>
|   |   |   |-- <assignment-statement>
|   |   |   |   |-- <variable>
|   |   |   |   |   |-- IDENT(akumulasi)
|   |   |   |   |-- BECOMES
|   |   |   |-- <expression>
|   |   |   |   |-- <simple-expression>
|   |   |   |   |   |-- <term>
|   |   |   |   |   |   |-- <factor>
|   |   |   |   |   |   |   |-- IDENT(akumulasi)
|   |   |   |   |-- <additive-operator>
|   |   |   |   |   |-- PLUS
|   |   |   |-- <term>
|   |   |   |   |-- <factor>
|   |   |   |   |   |-- INTCON(15)
|   |   |-- SEMICOLON
|   |   |-- <statement>
|   |   |   |-- <assignment-statement>
|   |   |   |   |-- <variable>
|   |   |   |   |   |-- IDENT(batas)
|   |   |   |   |-- BECOMES
|   |   |   |-- <expression>
|   |   |   |   |-- <simple-expression>
|   |   |   |   |   |-- <term>
|   |   |   |   |   |   |-- <factor>
|   |   |   |   |   |   |   |-- IDENT(batas)
|   |   |   |   |-- <additive-operator>
|   |   |   |   |   |-- MINUS
|   |   |   |-- <term>
|   |   |   |   |-- <factor>
|   |   |   |   |   |-- INTCON(1)
|   |   |-- SEMICOLON
|   |-- ENDSY
|-- SEMICOLON
|-- ENDSY
|-- PERIOD

===== DECORATED AST =====
|-- ProgramNode(name: 'TestTiga') {type: unknown, tabIndex: -1, lev: -1}
|   |-- Declarations
|   |   |-- TypeDeclNode(name: 'DataSiswa') {type: record, tabIndex: 41, lev: 0}
|   |   |   |-- RecordTypeNode {type: record, tabIndex: 2, lev: 0}
|   |   |   |   |-- VarDeclNode(names: [nomorInduk]) {type: integer, tabIndex: 38, lev:
1}
|   |   |   |   |   |-- NamedTypeNode(name: 'integer') {type: integer, tabIndex: -1,
lev: 1}
|   |   |   |   |   |   |-- VarDeclNode(names: [kodeKelas]) {type: char, tabIndex: 39, lev: 1}
|   |   |   |   |   |   |   |-- NamedTypeNode(name: 'char') {type: char, tabIndex: -1, lev: 1}
|   |   |   |   |-- VarDeclNode(names: [nilaiAkhir]) {type: real, tabIndex: 40, lev: 1}
|   |   |   |   |   |-- NamedTypeNode(name: 'real') {type: real, tabIndex: -1, lev: 1}
|   |   |-- VarDeclNode(names: [siswa]) {type: record, tabIndex: 42, lev: 0}
|   |   |   |-- NamedTypeNode(name: 'DataSiswa') {type: record, tabIndex: 41, lev: 0}

```



```
|  \-- VarDeclNode(names: [i, batas, akumulasi]) {type: integer, tabIndex: 45,
lev: 0}
|      \-- NamedTypeNode(name: 'integer') {type: integer, tabIndex: -1, lev: 0}
|  \-- Block
|      \-- CompoundNode {type: unknown, tabIndex: -1, lev: 0}
|          \-- AssignNode {type: void, tabIndex: -1, lev: 0}
|              |  \-- Target
|              |      \-- VarNode(name: 'batas') {type: integer, tabIndex: 44, lev: 0}
|              |      \-- Value
|              |          \-- NumberNode(value: '10', kind: integer) {type: integer,
tabIndex: -1, lev: 0}
|              \-- AssignNode {type: void, tabIndex: -1, lev: 0}
|                  |  \-- Target
|                  |      \-- VarNode(name: 'akumulasi') {type: integer, tabIndex: 45, lev:
0}
|                  \-- Value
|                      \-- NumberNode(value: '0', kind: integer) {type: integer, tabIndex:
-1, lev: 0}
|          \-- ForNode(iterator: 'i', direction: to) {type: void, tabIndex: 43, lev:
0}
|              |  \-- StartValue
|              |      \-- NumberNode(value: '1', kind: integer) {type: integer, tabIndex:
-1, lev: 0}
|              |  \-- EndValue
|              |      \-- VarNode(name: 'batas') {type: integer, tabIndex: 44, lev: 0}
|              \-- LoopBlock
|                  \-- CompoundNode {type: unknown, tabIndex: -1, lev: 0}
|                      \-- AssignNode {type: void, tabIndex: -1, lev: 0}
|                          |  \-- Target
|                          |      \-- VarNode(name: 'akumulasi') {type: integer,
tabIndex: 45, lev: 0}
|                          \-- Value
|                              \-- BinOpNode(op: '+') {type: integer, tabIndex: -1,
lev: 0}
|                                  |  \-- Left
|                                  |      \-- VarNode(name: 'akumulasi') {type: integer,
tabIndex: 45, lev: 0}
|                                  \-- Right
|                                      \-- VarNode(name: 'i') {type: integer,
tabIndex: 43, lev: 0}
|          \-- WhileNode {type: void, tabIndex: -1, lev: 0}
|              |  \-- Condition
|              |      \-- BinOpNode(op: '<') {type: boolean, tabIndex: -1, lev: 0}
|              |          |  \-- Left
|              |          |      \-- VarNode(name: 'akumulasi') {type: integer, tabIndex:
45, lev: 0}
|              |          \-- Right
|              |              \-- NumberNode(value: '100', kind: integer) {type: integer,
tabIndex: -1, lev: 0}
|              \-- LoopBlock
|                  \-- CompoundNode {type: unknown, tabIndex: -1, lev: 0}
|                      \-- AssignNode {type: void, tabIndex: -1, lev: 0}
|                          |  \-- Target
|                          |      \-- VarNode(name: 'akumulasi') {type: integer,
tabIndex: 45, lev: 0}
|                          \-- Value
|                              \-- BinOpNode(op: '+') {type: integer, tabIndex: -1,
lev: 0}
|                                  |  \-- Left
|                                  |      \-- VarNode(name: 'akumulasi') {type: integer,
tabIndex: 45, lev: 0}
|                                  \-- Right
|                                      \-- NumberNode(value: '15', kind: integer)
{type: integer, tabIndex: -1, lev: 0}
|                      \-- AssignNode {type: void, tabIndex: -1, lev: 0}
|                          |  \-- Target
|                          |      \-- VarNode(name: 'batas') {type: integer, tabIndex:
44, lev: 0}
|                          \-- Value
|                              \-- BinOpNode(op: '-') {type: integer, tabIndex: -1,
```



```
lev: 0}
|-- Left
|  \-- VarNode(name: 'batas') {type: integer,
tabIndex: 44, lev: 0}
|-- Right
|  \-- NumberNode(value: '1', kind: integer)
{type: integer, tabIndex: -1, lev: 0}

===== SYMBOL TABLE =====
===== TAB =====
idx  identifier      obj      type      ref      nrm      lev      adr      link
-----
0    AND             keyword   void      null      1        0        0       -1
1    ARRAY           keyword   void      null      1        0        0       -1
2    BEGIN           keyword   void      null      1        0        0       -1
3    CASE            keyword   void      null      1        0        0       -1
4    CONST           keyword   void      null      1        0        0       -1
5    DIV             keyword   void      null      1        0        0       -1
6    DOWNT0          keyword   void      null      1        0        0       -1
7    DO              keyword   void      null      1        0        0       -1
8    ELSE            keyword   void      null      1        0        0       -1
9    END             keyword   void      null      1        0        0       -1
10   FOR             keyword   void      null      1        0        0       -1
11   FUNCTION        keyword   void      null      1        0        0       -1
12   IF             keyword   void      null      1        0        0       -1
13   MOD            keyword   void      null      1        0        0       -1
14   NOT            keyword   void      null      1        0        0       -1
15   OF             keyword   void      null      1        0        0       -1
16   OR             keyword   void      null      1        0        0       -1
17   PROCEDURE      keyword   void      null      1        0        0       -1
18   PROGRAM        keyword   void      null      1        0        0       -1
19   RECORD         keyword   void      null      1        0        0       -1
20   REPEAT         keyword   void      null      1        0        0       -1
21   INTEGER        type      integer   null      1        0        0       -1
22   REAL           type      real      null      1        0        0       -1
23   BOOLEAN        type      boolean   null      1        0        0       -1
24   CHAR           type      char      null      1        0        0       -1
25   STRING         type      string    null      1        0        0       -1
26   THEN          keyword   void      null      1        0        0       -1
27   TO            keyword   void      null      1        0        0       -1
28   TYPE          keyword   void      null      1        0        0       -1
29   UNTIL         keyword   void      null      1        0        0       -1
30   VAR           keyword   void      null      1        0        0       -1
31   WHILE         keyword   void      null      1        0        0       -1
32   TRUE          constant  boolean   null      1        0        0       -1
33   FALSE         constant  boolean   null      1        0        0       32
34   READ          procedure void      null      1        0        0       33
35   READLN        procedure void      null      1        0        0       34
36   WRITE         procedure void      null      1        0        0       35
37   WRITELN       procedure void      null      1        0        0       36
38   NOMORINDUK   variable integer   null      1        1        0       -1
39   KODEKELAS    variable char      null      1        1        1       38
40   NILAIAKHIR   variable real      null      1        1        2       39
41   DATASISWA    type      record    null      1        0        0       37
42   SISWA        variable record    null      1        0        0       41
43   I            variable integer   null      1        0        1       42
44   BATAS        variable integer   null      1        0        2       43
45   AKUMULASI    variable integer   null      1        0        3       44

===== ATAB =====
idx      xtyp      etyp      eref      low      high      elsz      size
-----
===== BTAB =====
idx      last      lpar      psze      vsze
-----
0        37        -1        0        0
1        40        -1        0        3
2        45        -1        0        4
```





BAB IV KESIMPULAN & SARAN

4.1 Kesimpulan

Pada milestone ini telah berhasil diimplementasikan semantic analysis pada compiler bahasa Arion. Tahap ini berfungsi untuk memastikan program tidak hanya benar secara sintaks, tetapi juga valid secara makna melalui proses pembentukan AST, pengelolaan symbol table, traversal menggunakan Visitor Function, serta pemeriksaan type compatibility dan scope identifier.

Melalui semantic analysis, kita mampu mendeteksi berbagai semantic error seperti penggunaan identifier yang belum dideklarasikan dan ketidakcocokan tipe data pada expression maupun assignment. Dengan demikian, semantic analysis menjadi tahap penting sebelum compiler memasuki proses code generation.

4.2 Saran

Pengembangan semantic analyzer masih dapat ditingkatkan dengan menambahkan semantic checking yang lebih lengkap, optimasi struktur AST dan symbol table, serta peningkatan kualitas pesan semantic error agar lebih informatif.



LAMPIRAN

Link Repository Github

<https://github.com/Faizalfadaa/JKW-Tubes-IF2224-2026>

Pembagian Tugas

NIM	Kontribusi
13524007	25%
13524039	25%
13524069	25%
13524097	25%

REFERENSI

Gambar Parse Tree dan Abstract Syntax Tree (AST)

<https://ruslanspivak.com/Isbasi-part7>

Penjelasan AST

<https://nus-cs3203.github.io/course-website/contents/basic-spa-requirements/abstract-syntax-tree.html>

"Semantic Analysis (ULiège)"

<https://people.montefiore.uliege.be/geurts/Cours/compil/2015/04-semantic-2015-2016.pdf>